

```

1 FUNCTION zswx_get_direct_manager .
2  *"-
3  ***"Local Interface:
4  *"  IMPORTING
5  *"    VALUE(IGNORE_ACTING) TYPE  CHAR1 DEFAULT ' '
6  *"    VALUE(EFFECTIVE_DATE) TYPE  BEGDA DEFAULT SY-DATUM
7  *"    VALUE(EXPLAIN) TYPE  CHAR1 DEFAULT ' '
8  *"    VALUE(BYPASS_RESTRICTIONS) TYPE  CHAR1 DEFAULT ' '
9  *"    VALUE(MAX_LEVELS) TYPE  INT1 DEFAULT 0
10 *"  EXPORTING
11 *"    VALUE(ACTING) TYPE  BOOLE
12 *"  TABLES
13 *"    ACTOR_TAB STRUCTURE  SWHACTOR
14 *"    AC_CONTAINER STRUCTURE  SWCONT
15 *"  CHANGING
16 *"    VALUE(MANAGER_INFO) TYPE  STRING OPTIONAL
17 *"    VALUE(CALL_NUMBER) TYPE  INTEGER OPTIONAL
18 *"    VALUE(DETAILS_TAB) TYPE  ZTSTRINGS OPTIONAL
19 *"  EXCEPTIONS
20 *"    NOBODY_FOUND
21 *"    WORKFLOW_POSITION_EXCEPTION
22 *"    NO_EMPLOYEE_POSITION
23 *"    NO_ORGUNIT_FOUND
24 *"    NO_CHIEFPOS_FOUND
25 *"    CIRCULAR_REFERENCE
26 *"    MAX_RECURSIVE
27  *"-
28  * 2018.08.22 CJ Poorter Charm 2000002091
29  * Reference manager position in PA0016 for Contractors (Australia only)
30  *"-
31
32  DATA BEGIN OF org_object OCCURS 0.
33  INCLUDE STRUCTURE swhactor.
34  DATA END OF org_object.
35
36  DATA BEGIN OF acting_tab OCCURS 0.
37  INCLUDE STRUCTURE swhactor.
38  DATA END OF acting_tab.
39
40  DATA: lv_pernr          TYPE persno,
41         lv_continue,
42         lv_position      TYPE objid,
43         lv_bukrs         LIKE p0001-bukrs,
44         lv_persg         LIKE p0001-persg,
45         lv_orgunit       TYPE objid,
46         lv_chiefpos      TYPE objid,
47         lv_reports_to_position TYPE objid,
48         lv_acting_vac_flag,
49         lv_vac_reports_to.
50
51  DATA: lv_constval LIKE zhp002-constval,
52         lv_new_info TYPE string,
53         lv_details  TYPE text80,
54         lv_sobid    LIKE hrp1001-sobid,
55         lv_nextorg  LIKE objec-objid.
56
57  * Check for excessive calls (circular references)
58  ADD 1 TO call_number.
59  IF call_number GT 20.
60    MESSAGE e010 WITH 'Too many recursive calls)'
61                      space space space

```

```

62         RAISING max_recursive.
63     ENDIF.
64
65     REFRESH: actor_tab.
66
67
68     * Set acting check override
69     gv_ignact = ignore_acting.
70
71     gv_explain = explain.
72     gt_details_tab[] = details_tab[].
73
74     * Set effective date
75     IF effective_date IS INITIAL.
76         effective_date = sy-datum.
77     ENDIF.
78     gv_effdate = effective_date.
79
80     * Read workflow positions to be ignored from ZHPD002
81     REFRESH rng_ignpos.
82     CLEAR: rng_ignpos.
83     * Add dummy position 99999999 to range so that
84     * it does not equate to "all positions"
85     rng_ignpos-option = 'EQ'.
86     rng_ignpos-sign = 'I'.
87     rng_ignpos-low = '99999999'.
88     APPEND rng_ignpos.
89     IF bypass_restrictions NE 'X'.
90
91         CLEAR: rng_ignpos, gv_wfignore.
92         rng_ignpos-option = 'EQ'.
93         rng_ignpos-sign = 'I'.
94         SELECT constval INTO lv_constval
95             FROM zhp002
96             WHERE refid = 'WORKFLOW_IGNORE_POSITION'
97                 AND begda LE sy-datum
98                 AND endda GE sy-datum
99             ORDER BY seqnr.
100         rng_ignpos-low = lv_constval.
101         APPEND rng_ignpos.
102     ENDSELECT.
103 ENDIF.
104
105 * Get the org-object under consideration.
106 swc_get_element ac_container 'ORG_OBJECT' org_object.
107 IF sy-subrc NE 0.
108     details_tab[] = gt_details_tab[].
109     MESSAGE e010 WITH space space space space
110         RAISING nobody_found.
111 ENDIF.
112
113 *-----*
114 * Get the users employee number and position
115 PERFORM get_ee_data      USING org_object
116                             effective_date
117                             CHANGING lv_pernr
118                                     lv_position
119                                     lv_acting_vac_flag
120                                     lv_vac_reports_to
121                                     lv_bukrs
122                                     lv_persg.

```

```

123
124 * If no employee position found, then raise error
125 IF lv_position IS INITIAL.
126     details_tab[] = gt_details_tab[].
127     MESSAGE e010 WITH 'Cannot determine position for employee'
128         lv_pernr space space
129         RAISING no_employee_position.
130 ENDIF.
131
132 IF gv_explain EQ 'X'.
133     PERFORM details_line TABLES gt_details_tab USING:
134         lv_pernr 'Position' 'S' lv_position,
135         space 'VacFlag (acting)' space lv_acting_vac_flag,
136         space 'VacFlag (report to)' space lv_vac_reports_to.
137 ENDIF.
138
139
140 IF gv_ignact = 'X'.
141     CLEAR: lv_acting_vac_flag,lv_vac_reports_to.
142 ENDIF.
143
144 CLEAR: lv_reports_to_position.
145 * If request was for an orgunit supervisor (not person or position)
146 * then the chief of the orgunit was already returned in lv_position
147 * by the get_ee_data call above
148 IF org_object-otype = 'O'.
149     lv_reports_to_position = lv_position.
150     lv_chiefpos = lv_position.
151 ENDIF.
152 *-----*
153 * Phase 0 - Contractors only: Find the manager position on IT0016
154 *-----*
155 IF lv_reports_to_position IS INITIAL
156 AND (lv_persg = '8' OR lv_persg = '9' ).
157 * Initially only for Australian company codes
158 * Now extended for all countries
159 * AND lv_bukrs+0(2) = 'AU'.
160     PERFORM get_contractor_supervisor
161         USING lv_pernr
162             effective_date
163             CHANGING lv_reports_to_position.
164     IF gv_explain EQ 'X'.
165         PERFORM details_line TABLES gt_details_tab USING:
166             lv_pernr 'Contract supervisor (IT0016)' 'S'
167                 lv_reports_to_position.
168     ENDIF.
169 ENDIF.
170
171
172
173 *-----*
174 * Phase 1 - Find the manager through the A002 reporting line
175 *-----*
176
177 IF lv_position IS NOT INITIAL
178 AND lv_reports_to_position IS INITIAL.
179     PERFORM get_reports_to_position USING lv_position
180                                         effective_date
181                                         CHANGING lv_reports_to_position.
182     IF gv_explain EQ 'X'.
183         PERFORM details_line TABLES gt_details_tab USING:

```

```

184         lv_pernr 'Reports To (A002)' 'S' lv_reports_to_position.
185     ENDIF.
186 ENDIF.
187
188 * Check if another position is acting
189 IF lv_reports_to_position IS NOT INITIAL
190 AND gv_ignact IS INITIAL.
191     PERFORM check_position_acting USING      lv_pernr
192                                             effective_date
193                                             CHANGING lv_reports_to_position
194                                             acting.
195 ENDIF.
196
197 * Check that position is filled with active employee (not self)
198 IF lv_reports_to_position IS NOT INITIAL.
199     PERFORM confirm_position TABLES actor_tab
200                                     USING lv_pernr
201                                     lv_reports_to_position
202                                     effective_date.
203 ENDIF.
204
205 * If valid manager found then exit
206 DESCRIBE TABLE actor_tab LINES counter.
207 IF counter > 0.
208     CONCATENATE 'A002=>' lv_reports_to_position INTO lv_new_info.
209     IF acting = 'X'.
210         CONCATENATE lv_new_info '(Acting)' INTO lv_new_info.
211     ENDIF.
212     CONCATENATE manager_info lv_new_info INTO manager_info.
213     IF gv_explain EQ 'X'.
214         PERFORM details_line TABLES gt_details_tab USING:
215             lv_pernr '=> Has manager (A002)' 'P' actor_tab-objid.
216     ENDIF.
217     details_tab[] = gt_details_tab[].
218     EXIT.
219 ENDIF.
220
221 *-----*
222 * Phase 2 - Find the manager through the Orgunit Chief Position
223 *-----*
224 CLEAR acting.
225
226 * Get all the orgunits and chiefs in the hierarchy above
227 * that the position belongs to (excluding own position)
228 * CP 16.03.2019 Added parameter max_levels to restrict
229 *         technical depth of evaluation path S-O-O
230 *         so that tracking to top of org is restricted
231 PERFORM get_chief_hierarchy TABLES actor_tab
232                                     USING lv_position
233                                     effective_date
234                                     max_levels.
235
236
237 * For next Chief Position found, confirm if a valid manager
238 LOOP AT actor_tab WHERE otype = 'S'.
239
240     IF gv_explain EQ 'X'.
241         PERFORM details_line TABLES gt_details_tab USING:
242             lv_position 'Try Chief Position' 'S' actor_tab-objid.
243     ENDIF.
244

```

```

245  * Check that chief position is not self
246      IF actor_tab-objid NE lv_position.
247          lv_chiefpos = actor_tab-objid.
248          IF gv_explain EQ 'X'.
249              PERFORM details_line TABLES gt_details_tab USING:
250                  space 'OK - selected' 'S' lv_chiefpos.
251          ENDIF.
252          EXIT.
253      ELSE.
254          IF gv_explain EQ 'X'.
255              PERFORM details_line TABLES gt_details_tab USING:
256                  space 'Ignored - own position' 'S' lv_position.
257          ENDIF.
258      ENDIF.
259  ENDLOOP.
260
261  IF sy-subrc NE 0.
262      details_tab[] = gt_details_tab[].
263      MESSAGE e010 WITH 'Cannot find next chief for position'
264                      lv_position space space
265                      RAISING no_chiefpos_found.
266  ENDIF.
267
268  * Check if another position is acting for this chief position
269  IF gv_ignact IS INITIAL.
270      PERFORM check_position_acting USING      lv_pernr
271                                              effective_date
272                                              CHANGING lv_chiefpos
273                                              acting.
274  ENDIF.
275
276  * Check that position is filled with active employee (not self)
277  PERFORM confirm_position TABLES actor_tab
278                          USING lv_pernr
279                          lv_chiefpos
280                          effective_date.
281
282
283  * If valid manager found then exit
284  DESCRIBE TABLE actor_tab LINES counter.
285  IF counter > 0.
286      CONCATENATE 'B012=>' lv_chiefpos INTO lv_new_info.
287      IF acting = 'X'.
288          CONCATENATE lv_new_info '(Acting)' INTO lv_new_info.
289      ENDIF.
290      CONCATENATE manager_info lv_new_info INTO manager_info.
291
292      IF gv_explain EQ 'X'.
293          PERFORM details_line TABLES gt_details_tab USING:
294              lv_pernr '=> Has manager (chief)' 'P' actor_tab-objid.
295      ENDIF.
296      details_tab[] = gt_details_tab[].
297      EXIT.
298  ENDIF.
299
300  IF org_object-otype = 'O'. "OrgUnit
301  * Get superior orgunit
302      SELECT SINGLE sobid INTO lv_sobid
303      FROM hrp1001
304      WHERE plvar = '01'
305            AND otype = 'O'

```

```

306      AND objid = org_object-objid
307      AND rsign = 'A'
308      AND relat = '002'
309      AND subty = 'A002'
310      AND begda LE effective_date
311      AND endda GE effective_date
312      AND sclas = 'O'.
313  IF sy-subrc NE 0.
314      MESSAGE e010 WITH 'Cannot find superior orgunit for'
315                      org_object-objid space space
316                      RAISING no_orgunit_found.
317      RETURN.
318  ELSE.
319      lv_nextorg = lv_sobid(8).
320  ENDIF.
321  IF gv_explain EQ 'X'.
322      PERFORM details_line TABLES gt_details_tab USING:
323          lv_pernr 'Recursive Call' 'O' lv_nextorg.
324  ENDIF.
325  PERFORM recursive_call_org TABLES actor_tab
326                          USING lv_nextorg
327                          effective_date
328                          max_levels
329                          CHANGING acting
330                          manager_info
331                          call_number.
332  ELSE.
333  * If manager position is not valid, find the manager of that chief
334      IF gv_explain EQ 'X'.
335          PERFORM details_line TABLES gt_details_tab USING:
336              lv_pernr 'Recursive Call' 'S' lv_chiefpos.
337      ENDIF.
338      PERFORM recursive_call TABLES actor_tab
339                          USING lv_chiefpos
340                          effective_date
341                          max_levels
342                          CHANGING acting
343                          manager_info
344                          call_number.
345
346
347  ENDIF.
348  * Check if we have found some manager.
349  details_tab[] = gt_details_tab[].
350  DESCRIBE TABLE actor_tab LINES counter.
351  IF counter = 0.
352      MESSAGE e010 WITH space space space space
353                      RAISING nobody_found.
354  ELSE.
355      IF gv_explain EQ 'X'.
356          PERFORM details_line TABLES gt_details_tab USING:
357              lv_pernr '=> Has manager (chief)' 'P' actor_tab-objid.
358      ENDIF.
359      EXIT.
360  ENDIF.
361
362
363  ENDFUNCTION.

```

Subroutines:		Line	Page
CHECK_POSITION_ACTING.....	LZSWXAU12	191	4
	LZSWXAU12	270	5
CONFIRM_POSITION.....	LZSWXAU12	199	4
	LZSWXAU12	277	5
DETAILS_LINE.....	LZSWXAU12	133	3
	LZSWXAU12	133	3
	LZSWXAU12	133	3
	LZSWXAU12	165	3
	LZSWXAU12	183	3
	LZSWXAU12	214	4
	LZSWXAU12	241	4
	LZSWXAU12	249	5
	LZSWXAU12	255	5
	LZSWXAU12	293	5
	LZSWXAU12	322	6
	LZSWXAU12	335	6
	LZSWXAU12	356	6
GET_CHIEF_HIERARCHY.....	LZSWXAU12	231	4
GET_CONTRACTOR_SUPERVISOR.....	LZSWXAU12	160	3
GET_EE_DATA.....	LZSWXAU12	115	2
GET_REPORTS_TO_POSITION.....	LZSWXAU12	179	3
RECURSIVE_CALL.....	LZSWXAU12	338	6
RECURSIVE_CALL_ORG.....	LZSWXAU12	325	6

Messages:		Line	Page
010.....	LZSWXAU12	60	1
	LZSWXAU12	109	2
	LZSWXAU12	127	3
	LZSWXAU12	263	5
	LZSWXAU12	314	6
	LZSWXAU12	352	6

DATABASE OPERATIONS

SELECT:	Line	Page
HRP1001..... LZSWXAU12	303	5
ZHPD002..... LZSWXAU12	95	2


```

1  FUNCTION-POOL zswxa      MESSAGE-ID wz.
2  TYPE-POOLS shlp.
3  TYPE-POOLS f4typ.  "Brücke zu alten F4-Bausteinen
4  TABLES: ddshdefsh.  "Tabelle der Default-Suchhilfen
5
6  INCLUDE rswuincl.
7  INCLUDE <cntain>.
8
9  * Table SWXFORMABS represents a "notice of absence form".
10 TABLES: swxformabs,
11          pa0001.
12
13 DATA: action(4).
14
15 * Constants for actions.
16 DATA: action_create(4) VALUE 'CREA', action_update(4) VALUE 'UPDA',
17        action_display(4) VALUE 'DISP', action_approve(4) VALUE 'APPR'.
18 DATA: answer_abort VALUE 'A'.
19 DATA: ok-code(4).
20 DATA: save_okcode(4).
21
22 DATA: BEGIN OF functab OCCURS 2,
23        function(4),
24 END OF functab.
25
26 * Processing state of the notice of absence.
27 DATA: processing_state(10).
28
29 * Constants for the processing state abbreviation.
30 DATA: state_approved LIKE swxformabs-procstate VALUE 'A',
31        state_rejected LIKE swxformabs-procstate VALUE 'R',
32        state_new LIKE swxformabs-procstate VALUE 'N'.
33
34 * Dummy data concerning the dynpro.
35
36 DATA: persdata_frame, absencedata_frame, recreation_button,
37        specialholiday_button, sign_frame.
38 DATA recreation_button1.
39 DATA recreation_button2.
40 DATA recreation_button3.
41 DATA specialholiday_button1.
42 DATA specialholiday_button2.
43 DATA specialholiday_button3.
44
45 *- toolbar...
46 INCLUDE lswxalcl.
47 DATA: g_toolbox_mode.
48 *- demoing the modeless toolbox...
49 DATA: my_modeless_toolbox TYPE REF TO cl_wapi_wf_toolbox,
50        my_modeless_toolbox_handler TYPE REF TO wf_toolbox_event_handler.
51 *- demoing the inplace toolbox (small version)
52 DATA: my_small_container TYPE REF TO cl_gui_custom_container,
53        my_small_toolbox TYPE REF TO cl_wapi_wf_toolbox,
54        my_small_toolbox_handler TYPE REF TO wf_toolbox_event_handler.
55 *- demoing the inplace toolbox (reduced one button version)
56 DATA: my_button_container TYPE REF TO cl_gui_custom_container,
57        my_button_toolbox TYPE REF TO cl_wapi_wf_toolbox,
58        my_button_toolbox_handler TYPE REF TO wf_toolbox_event_handler.
59 *- demoing the inplace toolbox (large version on separate tabstrip)
60 DATA: my_big_container TYPE REF TO cl_gui_custom_container,
61        my_big_toolbox TYPE REF TO cl_wapi_wf_toolbox,

```

```

62         my_big_toolbox_handler TYPE REF TO wf_toolbox_event_handler.
63
64 *- tabstrip on screen 610
65 CONTROLS: strip TYPE TABSTRIP.
66 DATA:  gv_subscreen           LIKE sy-dynnr.
67
68 *- data for searchhelp exit
69 DATA %shlpname LIKE dd30v-shlpname.      " für Übergabe an Callback-Form
70 " von F4TOOL_SUBSHLP_BY_HOTKEY
71 CONSTANTS: par%domname LIKE ddshfprop-fieldname VALUE 'DOMNAME',
72             par%value LIKE ddshfprop-fieldname VALUE 'VALUE',
73             par%text LIKE ddshfprop-fieldname VALUE 'TEXT',
74             par%_low LIKE ddshfprop-fieldname VALUE 'LOW',
75             par%_high LIKE ddshfprop-fieldname VALUE 'HIGH',
76             par%_text LIKE ddshfprop-fieldname VALUE 'TEXT',
77             par%rollname LIKE ddshfprop-fieldname VALUE 'ROLLNAME',
78             par%tabname LIKE ddshfprop-fieldname VALUE 'TABNAME'.
79
80 DATA:  gv_curr_pernr LIKE pa0105-pernr,
81         gv_curr_plans LIKE pa0001-plans,
82         gv_act_plans  LIKE pa0001-plans,
83         gv_lines TYPE i.
84
85 DATA: gt_users LIKE agr_users OCCURS 1000 WITH HEADER LINE.
86
87 DATA: BEGIN OF gt_mss_employees OCCURS 1000,
88         pernr LIKE pa0105-pernr,
89         END OF gt_mss_employees.
90
91 DATA: gt_act_tab LIKE zppa014 OCCURS 10 WITH HEADER LINE,
92         gt_act_tab_struct like struct OCCURS 10 WITH HEADER LINE.
93
94 DATA: gv_actingname TYPE pad_vorna.
95 DATA: gs_mgrname TYPE pa0002.
96 DATA: gv_begda(10),
97         gv_endda(10),
98         gv_users_ccgrp LIKE zppa000-ccgrp.
99
100 DATA: counter TYPE i.
101 DATA: eval_path LIKE gdstr-wegid.
102 DATA: eval_depth LIKE hrrhas-tdepth.
103
104 DATA: gs_p2003 LIKE p2003,
105         gs_RETURN1 LIKE BAPIRETURN1,
106         gs_RETURN LIKE BAPIRETURN.
107
108 data: gt_prop_values type TABLE OF pprop,
109       wa_prop type pprop,
110       gv_portal.
111
112 data: gv_ignact(1).
113
114 * Range to determine special positions to ignore in workflow
115 * Read from table ZHPD002 with REFID=WORKFLOW_IGNORE_POSITION
116 * RT17395
117 ranges: rng_ignpos for hrp1000-objid.
118
119 data: gv_wfignore,
120       gv_effdate type d,
121       gv_explain,
122 *{ REPLACE S01K900038

```

```
123  *\      gt_details_tab type tstrings.  
124      gt_details_tab type Ztstrings.  
125  *}  REPLACE
```

Subroutines:	Line	Page
ABORT.....	LSWXALCL	19
	LSWXALCL	21
	LSWXALCL	23

```

1  *-----*
2  ***INCLUDE LSWXAF01 .
3  *-----*
4  *&-----*
5  *&      Form  FORM_PROCESS
6  *&-----*
7  *      text
8  *-----*
9  *  —>  p1      text
10 *  <—  p2      text
11 *-----*
12 FORM form_process.
13
14     save_okcode = ok-code.
15     CLEAR ok-code.
16
17     CASE save_okcode.
18         WHEN 'EXIT'.
19             PERFORM form_exit.
20         WHEN 'BACK'.
21             PERFORM form_exit.
22         WHEN 'ABOR'.
23             PERFORM abort.
24         WHEN 'SAVE'.
25             PERFORM form_save_and_exit.
26         WHEN 'APPR'.
27             PERFORM form_approve.
28         WHEN 'REJE'.
29             PERFORM form_reject.
30         WHEN 'SSC1'.
31             strip-activetab = 'SSC1'.
32         WHEN 'SSC2'.
33             strip-activetab = 'SSC2'.
34     ENDCASE.
35
36 ENDFORM.                                " PROCESS_FORM
37
38 *&-----*
39 *&      Form  FORM_CHECK_SAVED
40 *&-----*
41 *      text
42 *-----*
43 *  —>  p1      text
44 *  <—  p2      text
45 *-----*
46 FORM form_check_saved.
47
48     DATA: textline1(35), textline2(35), titel(35), answer.
49
50     titel = 'Bearbeitung beenden'(a01).
51     textline1 = 'Formular nicht gesichert.'(a02).
52     textline2 = 'Wollen Sie es sichern?'(a03).
53
54     CALL FUNCTION 'POPUP_TO_CONFIRM_STEP'
55         EXPORTING
56             defaultoption = 'Y'
57             textline1      = textline1
58             textline2      = textline2
59             titel          = titel
60     IMPORTING
61         answer            = answer.

```

```

62
63 CASE answer.
64     WHEN answer_yes.
65         PERFORM form_save_and_exit.
66     WHEN answer_no.
67         PERFORM abort.
68     WHEN answer_abort.
69         EXIT.
70     WHEN OTHERS.
71         MESSAGE e800.
72     *      Unbekannter Fehler aufgetreten.
73     ENDCASE.
74
75 ENDFORM.                                " FORM_CHECK_SAVED
76
77 *-----*
78 *      Form  ABORT
79 *-----*
80 *      text
81 *-----*
82 *  —>  p1      text
83 *  <—  p2      text
84 *-----*
85 FORM abort.
86
87     ok-code = 'ABOR'.
88     SET SCREEN 0.
89     LEAVE SCREEN.
90
91 ENDFORM.                                " ABORT
92
93 *-----*
94 *      Form  FORM_SAVE_and_exit
95 *-----*
96 *      text
97 *-----*
98 *  —>  p1      text
99 *  <—  p2      text
100 *-----*
101 FORM form_save_and_exit.
102
103 * Store.
104 CASE action.
105     WHEN action_create.
106         INSERT swxformabs.
107     WHEN action_update.
108         UPDATE swxformabs.
109     WHEN action_approve.
110         UPDATE swxformabs.
111     WHEN OTHERS.
112         MESSAGE e800.
113     *      Unbekannter Fehler aufgetreten.
114     ENDCASE.
115
116     IF sy-subrc NE 0.
117         MESSAGE e002.
118     *      Error when updating / inserting record.
119     ELSE.
120         MESSAGE s003.
121     *      UPDATE SUCCEEDED.
122

```

```

123 *   Exit
124     PERFORM exit.
125
126     ENDIF.
127
128 ENDFORM.                                " FORM_SAVE
129
130 *&-----*
131 *&       Form  FORM_EXIT
132 *&-----*
133 *       text
134 *-----*
135 *  -->  p1       text
136 *  <--  p2       text
137 *-----*
138 FORM form_exit.
139
140     CASE action.
141         WHEN action_create.
142             PERFORM form_check_saved.
143         WHEN action_update.
144             PERFORM form_check_saved.
145         WHEN action_approve.
146             PERFORM form_check_approved.
147         WHEN OTHERS.
148 *       Leave the whole program.
149             PERFORM exit.
150     ENDCASE.
151 ENDFORM.                                " FORM_EXIT
152
153 *&-----*
154 *&       Form  FORM_CHECK_APPROVED
155 *&-----*
156 *       text
157 *-----*
158 *  -->  p1       text
159 *  <--  p2       text
160 *-----*
161 FORM form_check_approved.
162
163     DATA: textline1(40), textline2(40), titel(40), answer.
164
165 * Check if form as been approved or rejected.
166 IF  swxformabs-procstate = state_approved
167 OR  swxformabs-procstate = state_rejected.
168     PERFORM form_save_and_exit.
169 ELSE.
170 * Ask if processing should be aborted.
171     titel = text-c01.
172     textline1 = text-c02.
173     textline2 = text-c03.
174
175     CALL FUNCTION 'POPUP_TO_CONFIRM_STEP'
176     EXPORTING
177         defaultoption = 'Y'
178         textline1      = textline1
179         textline2      = textline2
180         titel          = titel
181     IMPORTING
182         answer         = answer.
183

```

```

184     CASE answer.
185         WHEN answer_yes.
186             PERFORM abort.
187         WHEN answer_no.
188             EXIT.
189         WHEN answer_abort.
190             EXIT.
191         WHEN OTHERS.
192             MESSAGE e800.
193     *   Unbekannter Fehler aufgetreten.
194     ENDCASE.
195     ENDIF.
196     ENDFORM.                                " FORM_CHECK_APPROVED
197
198 *-----*
199 *&      Form   EXIT
200 *-----*
201 *      text
202 *-----*
203 *  —>  p1      text
204 *  <—  p2      text
205 *-----*
206     FORM exit.
207
208     SET SCREEN 0.
209     LEAVE SCREEN.
210
211     ENDFORM.                                " EXIT
212
213 *-----*
214 *&      Form   FORM_APPROVE
215 *-----*
216 *      text
217 *-----*
218 *  —>  p1      text
219 *  <—  p2      text
220 *-----*
221     FORM form_approve.
222
223     * Sign the form.
224     swxformabs-approvdate = sy-datum.
225     swxformabs-approvby = sy-uname.
226     * Update the processing state and the form layout.
227     swxformabs-procstate = state_approved.
228     PERFORM form_layout.
229     MESSAGE s006.
230     * Antrag genehmigt.
231
232     ENDFORM.                                " FORM_APPROVE
233
234 *-----*
235 *&      Form   FORM_REJECT
236 *-----*
237 *      text
238 *-----*
239 *  —>  p1      text
240 *  <—  p2      text
241 *-----*
242     FORM form_reject.
243
244     * Clear the "approval signature" of the form.

```



```

245 CLEAR swxformabs-approvdate.
246 CLEAR swxformabs-approvby.
247 * Update the processing state and the form layout.
248 swxformabs-procstate = state_rejected.
249 PERFORM form_layout.
250 MESSAGE s008.
251 * Antrag nicht genehmigt.
252
253 ENDFORM. " FORM_REJECT
254
255 *&-----*
256 *& Form FORM_LAYOUT
257 *&-----*
258 * text
259 *-----*
260 * —> p1 text
261 * <— p2 text
262 *-----*
263 FORM form_layout.
264 *** Set the holiday buttons.
265
266 * Clear the special-holiday/recreation buttons.
267 CLEAR: specialholiday_button1, recreation_button1,
268 specialholiday_button2, recreation_button2,
269 specialholiday_button3, recreation_button3.
270
271 * Set the holiday buttons on the dynpro.
272 IF swxformabs-holidaytp1 = 'S'.
273 specialholiday_button1 = 'X'.
274 ELSE.
275 recreation_button1 = 'X'.
276 ENDIF.
277
278 IF swxformabs-holidaytp2 = 'S'.
279 specialholiday_button2 = 'X'.
280 ELSE.
281 recreation_button2 = 'X'.
282 ENDIF.
283
284 IF swxformabs-holidaytp3 = 'S'.
285 specialholiday_button3 = 'X'.
286 ELSE.
287 recreation_button3 = 'X'.
288 ENDIF.
289
290 *** Set the processing state of the form.
291
292 CASE swxformabs-procstate.
293 WHEN 'A'. "approved
294 processing_state = text-b01.
295 WHEN 'R'. "rejected
296 processing_state = text-b02.
297 WHEN 'N'. "new
298 processing_state = text-b03.
299 WHEN OTHERS.
300 * This case should not occur.
301 MESSAGE e800.
302 * Unbekannter Fehler aufgetreten.
303 ENDCASE.
304
305 ENDFORM. " FORM_LAYOUT

```

```

306
307 *&-----*
308 *&      Form  DYNPRO_051_INIT
309 *&-----*
310 *      text
311 *-----*
312 *  →  p1      text
313 *  ←  p2      text
314 *-----*
315 FORM dynpro_051_init.
316
317 *- local data for toolbox...
318   DATA: l_repid LIKE sy-repid,
319         l_dynnr LIKE sy-dynnr,
320         l_func  TYPE i.
321
322 *- start of demo workflow toolbox
323   l_repid = sy-repid.
324   l_dynnr = '0011'.
325 *- directly before we call the screen, we create a toolbox instance...
326   IF my_big_container IS INITIAL.
327     CREATE OBJECT my_big_container
328     EXPORTING
329       container_name = 'TOOLBOX_CONTAINER_BIG'
330       repid          = l_repid
331       dynnr          = l_dynnr
332     EXCEPTIONS
333       OTHERS         = 1.
334   ENDIF.
335
336 *- if we want to enable forwarding and resubmission, we have to
337 * create an event handler...
338   IF my_big_toolbox_handler IS INITIAL.
339     CREATE OBJECT my_big_toolbox_handler.
340   ENDIF.
341
342 *- now we create our toolbox in the style we want to have it...
343   IF my_big_toolbox IS INITIAL.
344     ADD cl_wapi_wf_toolbox=>c_function_forward available TO l_func.
345     ADD cl_wapi_wf_toolbox=>c_function_resubmit_available TO l_func.
346
347     CREATE OBJECT my_big_toolbox
348     EXPORTING
349       i_container      = my_big_container
350       i_repid          = l_repid
351       i_dynnr          = l_dynnr
352       i_mode           = cl_wapi_wf_toolbox=>c_mode_inplace_with_in
fo
353       i_add_functions  = l_func
354       i_add_separator  = cl_wapi_wf_toolbox=>c_show_separator
355       i_button_style   = cl_wapi_wf_toolbox=>c_button_style_with_te
xt
356     EXCEPTIONS
357       wrong_mode       = 1
358       container_missing = 2
359       error_workitem_read = 3
360       OTHERS           = 4.
361   IF sy-subrc EQ 0.
362     SET HANDLER my_big_toolbox_handler->on_user_took_workitem_away
363     FOR my_big_toolbox.
364   ENDIF.

```

```

365 ELSE.
366     CALL METHOD my_big_toolbox->refresh
367     EXCEPTIONS
368     OTHERS = 1.
369 ENDIF.
370
371 *- end of demo workflow toolbox
372
373 ENDFORM.                                " DYNPRO_051_INIT
374 *&-----*
375 *&      Form  SCREEN_011
376 *&-----*
377 *      text
378 *-----*
379 *  →  p1      text
380 *  ←  p2      text
381 *-----*
382 FORM screen_011.
383     CASE strip-activetab.
384         WHEN 'SSC1'.
385             g_subscreen      = '0050'.
386         WHEN 'SSC2'.
387             g_subscreen      = '0051'.
388     ENDCASE.
389 ENDFORM.                                " SCREEN_011
390 *&-----*
391 *&      Form  dynpro_010_init
392 *&-----*
393 *      text
394 *-----*
395 *  →  p1      text
396 *  ←  p2      text
397 *-----*
398 FORM dynpro_010_init .
399     DATA lv_name TYPE swl_aagt_n.
400     IF swxformabs-holidaytp1 IS INITIAL.
401         swxformabs-holidaytp1 = 'R'.
402     ENDIF.
403     IF swxformabs-holidaytp2 IS INITIAL.
404         swxformabs-holidaytp2 = 'R'.
405     ENDIF.
406     IF swxformabs-holidaytp3 IS INITIAL.
407         swxformabs-holidaytp3 = 'R'.
408     ENDIF.
409
410     CASE action.
411         WHEN action_create.
412             SET TITLEBAR 'CRE'.
413         WHEN action_update.
414             SET TITLEBAR 'UPD'.
415         WHEN action_display.
416             SET TITLEBAR 'DIS'.
417         WHEN action_approve.
418             SET TITLEBAR 'APP'.
419         WHEN OTHERS.
420             MESSAGE e800.
421     ENDCASE.
422
423 REFRESH functab.
424 CASE action.
425     WHEN action_create.

```

```

426         functab-function = 'APPR'.
427         APPEND functab.
428         functab-function = 'REJE'.
429         APPEND functab.
430     WHEN action_update.
431         functab-function = 'APPR'.
432         APPEND functab.
433         functab-function = 'REJE'.
434         APPEND functab.
435     WHEN action_display.
436         functab-function = 'SAVE'.
437         APPEND functab.
438         functab-function = 'APPR'.
439         APPEND functab.
440         functab-function = 'REJE'.
441         APPEND functab.
442     WHEN action_approve.
443         REFRESH functab.
444         functab-function = 'SAVE'.
445         APPEND functab.
446     WHEN OTHERS.
447         MESSAGE e800.
448     ENDCASE.
449     SET PF-STATUS 'FORMSTAT' EXCLUDING functab.
450
451 ENDFORM.                                " dynpro_010_init
452 *&-----*
453 *&      Form  dynpro_050_init
454 *&-----*
455 *      text
456 *-----*
457 *  →  p1      text
458 *  ←  p2      text
459 *-----*
460 FORM dynpro_050_init .
461     IF action = action_display OR action = action_approve.
462 *      All fields are set insensitive.
463         LOOP AT SCREEN.
464             screen-input = 0.
465             MODIFY SCREEN.
466         ENDLOOP.
467     ELSEIF action = action_create OR action = action_update.
468 *      The read-only fields are set insensitive.
469         LOOP AT SCREEN.
470             IF screen-group1 = 'RDO'.
471                 screen-input = 0.
472             ENDIF.
473             MODIFY SCREEN.
474         ENDLOOP.
475 *- set default values
476         IF swxformabs-firstday1 IS INITIAL.
477             swxformabs-firstday1 = sy-datum.
478         ENDIF.
479         IF swxformabs-lastday1 IS INITIAL.
480             swxformabs-lastday1 = sy-datum.
481         ENDIF.
482     ELSE.
483         MESSAGE e800.
484     ENDIF.
485 *- handle toolbox demos
486     CASE g_toolbox_mode.

```

```

487     WHEN 'S'.
488         LOOP AT SCREEN.
489             IF screen-name = 'SMALLTOOLBOXFRAME'.
490                 screen-invisible = 0.
491                 screen-active = 1.
492                 MODIFY SCREEN.
493             ENDIF.
494         ENDLOOP.
495     WHEN 'R'.
496         LOOP AT SCREEN.
497             IF screen-group1 = 'FRM'.
498                 screen-invisible = 1.
499             *         screen-active = 1.
500                 MODIFY SCREEN.
501             ENDIF.
502         ENDLOOP.
503     WHEN OTHERS.
504     ENDCASE.
505 ENDFORM.                                " dynpro_050_init
506 *&-----*
507 *&         Form  get_number
508 *&-----*
509 *         text
510 *-----*
511 *         <--P_FORMNUMBER  text
512 *-----*
513 FORM get_number  CHANGING lv_formnumber TYPE swx_fnumb.
514 * Generate a new number for the form.
515 CALL FUNCTION 'NUMBER_GET_NEXT'
516     EXPORTING
517         nr_range_nr           = '01'
518         object                = 'FORMABSENC'
519         quantity              = '1'
520         subobject             = ' '
521         toyear                = '0000'
522     IMPORTING
523         number                = lv_formnumber
524     EXCEPTIONS
525         interval_not_found    = 01
526         number_range_not_intern = 02
527         object_not_found      = 03
528         quantity_is_0         = 04.
529 * If range does not exist, create a new one.
530 IF sy-subrc = 1.
531     CALL FUNCTION 'SWU_NUMBER_RANGE_CREATE'
532         EXPORTING
533             object                = 'FORMABSENC'
534             nr_range_nr1         = '01'
535             quantity              = 1
536         IMPORTING
537             number                = lv_formnumber
538         EXCEPTIONS
539             range_already_exists = 01
540             object_not_found      = 02
541             system_error         = 03
542             OTHERS                = 04.
543     ENDIF.
544 ENDFORM.                                " get_number
545 *&-----*
546 *&         Form  creation_initialize
547 *&-----*

```

```

548 *      text
549 *-----*
550 *  →  p1      text
551 *  ←  p2      text
552 *-----*
553 FORM creation_initialize USING lv_formnumber TYPE swx_fnumb.
554   DATA lv_name TYPE swl_aagt_n.
555   CLEAR swxformabs.
556   swxformabs-formnumber = lv_formnumber.
557   swxformabs-createdate = sy-datum.
558   swxformabs-createdby = sy-uname.
559   swxformabs-procstate = state_new.
560   CALL FUNCTION 'SWL_AGENTS_NAME_DETERMIN'
561     EXPORTING
562       agent          = sy-uname
563       check_hide_agent = space
564     IMPORTING
565       agent_name      = lv_name.
566   swxformabs-name = lv_name.
567 ENDFORM.                                " creation_initialize
568 *&-----*
569 *&      Form initialize
570 *&-----*
571 *      text
572 *-----*
573 *  →  p1      text
574 *  ←  p2      text
575 *-----*
576 FORM initialize .
577   CLEAR ok-code.
578   CLEAR g_toolbox_mode.
579 ENDFORM.                                " initialize
580 *&-----*
581 *&      Form check_acting_manager
582 *&-----*
583 *      text
584 *-----*
585 *  →P_ACTOR_TAB text
586 *-----*
587 FORM check_acting_manager TABLES actor_tab STRUCTURE swhactor
588   USING p_pernr
589         p_datum
590   CHANGING acting.
591
592   DATA BEGIN OF position_tab OCCURS 0.
593     INCLUDE STRUCTURE swhactor.
594   DATA END OF position_tab.
595
596   DATA BEGIN OF acting_tab OCCURS 0.
597     INCLUDE STRUCTURE swhactor.
598   DATA END OF acting_tab.
599
600   DATA BEGIN OF add_actor_tab OCCURS 0.
601     INCLUDE STRUCTURE swhactor.
602   DATA END OF add_actor_tab.
603
604   DATA BEGIN OF reports_to_tab OCCURS 0.
605     INCLUDE STRUCTURE swhactor.
606   DATA END OF reports_to_tab.
607
608   DATA BEGIN OF chief_tab OCCURS 0.

```

```

609          INCLUDE STRUCTURE swfactor.
610 DATA END OF chief_tab.
611
612 DATA: counter TYPE i.
613 DATA: eval_path LIKE gdstr-wegid.
614 DATA: eval_depth LIKE hrrhas-tdepth.
615 DATA: lv_usrid LIKE pa0105-usrid,
616       lv_own_user.
617 DATA: lt_actor_tab LIKE swfactor OCCURS 10 WITH HEADER LINE,
618       lt_ac_container LIKE swcont OCCURS 10 WITH HEADER LINE.
619
620 lv_own_user = 'N'.
621 * check for acting relationship from the chief's position
622 CLEAR add_actor_tab. REFRESH add_actor_tab.
623 LOOP AT actor_tab.
624
625 * Get the position
626 CLEAR position_tab. REFRESH position_tab.
627 CALL FUNCTION 'RH_STRUC_GET'
628     EXPORTING
629         act_otype           = actor_tab-otype
630         act_objid           = actor_tab-objid
631         act_wegid           = 'B008'
632         act_plvar           = ' '
633         act_begda           = p_datum
634         act_endda           = p_datum
635         act_tdepth          = 2 " no recursion!
636         authority_check     = ' '
637     TABLES
638         result_tab          = position_tab
639     EXCEPTIONS
640         no_plvar_found      = 01
641         no_entry_found      = 02.
642
643 IF sy-subrc EQ 0.
644     LOOP AT position_tab.
645         CLEAR acting_tab. REFRESH acting_tab.
646         CALL FUNCTION 'RH_STRUC_GET'
647             EXPORTING
648                 act_otype           = position_tab-otype
649                 act_objid           = position_tab-objid
650                 act_wegid           = 'ZAZ15'
651                 act_plvar           = ' '
652                 act_begda           = p_datum
653                 act_endda           = p_datum
654                 act_tdepth          = 2 " no recursion!
655                 authority_check     = ' '
656             TABLES
657                 result_tab          = acting_tab
658             EXCEPTIONS
659                 no_plvar_found      = 01
660                 no_entry_found      = 02.
661 IF sy-subrc NE 0.
662 * Check for Z08 relationship
663 CLEAR acting_tab. REFRESH acting_tab.
664 CALL FUNCTION 'RH_STRUC_GET'
665     EXPORTING
666         act_otype           = position_tab-otype
667         act_objid           = position_tab-objid
668         act_wegid           = 'ZAZ08'
669         act_plvar           = ' '

```

```

670         act_begda      = p_datum
671         act_endda      = p_datum
672         act_tdepth     = 2          " no recursion!
673         authority_check = ' '
674     TABLES
675         result_tab      = add_actor_tab
676     EXCEPTIONS
677         no_plvar_found  = 01
678         no_entry_found  = 02.
679 ELSE.
680     LOOP AT acting_tab.
681 * Get holders
682     CLEAR add_actor_tab. REFRESH add_actor_tab.
683     CALL FUNCTION 'RH_STRUC_GET'
684     EXPORTING
685         act_otype      = acting_tab-otype
686         act_objid      = acting_tab-objid
687         act_wegid      = 'A008'
688         act_plvar      = ' '
689         act_begda      = p_datum
690         act_endda      = p_datum
691         act_tdepth     = 2          " no recursion!
692         authority_check = ' '
693     TABLES
694         result_tab      = add_actor_tab
695     EXCEPTIONS
696         no_plvar_found  = 01
697         no_entry_found  = 02.
698
699     ENDLOOP.          " AT acting_tab
700
701     ENDIF.
702     ENDLOOP.          " at position_tab.
703 ENDIF.
704
705     ENDLOOP.          " at actor_tab
706
707 * Remove the user themselves
708     LOOP AT add_actor_tab WHERE objid = p_pernr.
709         lv_own_user = 'Y'.
710     ENDLOOP.
711     IF lv_own_user = 'Y'.
712         DELETE add_actor_tab WHERE objid = p_pernr.
713
714     LOOP AT actor_tab.
715         SELECT SINGLE usrid FROM pa0105
716             INTO lv_usrid
717             WHERE pernr = actor_tab-objid
718             AND subty = '0001'
719             AND begda LE p_datum
720             AND endda GE p_datum.
721         lt_ac_container-element = 'ORG OBJECT'.
722         lt_ac_container-tab_index = '000001'.
723         lt_ac_container-elemlength = '001'.
724         CONCATENATE 'US' lv_usrid INTO lt_ac_container-value.
725         APPEND lt_ac_container.
726
727         CALL FUNCTION 'ZSWX_GET_DIRECT_MANAGER'
728         EXPORTING
729             effective_date = p_datum
730     TABLES

```



```

731         actor_tab      = lt_actor_tab
732         ac_container    = lt_ac_container
733     EXCEPTIONS
734         nobody_found    = 1
735         OTHERS           = 2.
736     IF sy-subrc = 0.
737         LOOP AT lt_actor_tab.
738             APPEND lt_actor_tab TO add_actor_tab.
739         ENDLOOP.
740
741     ENDIF.
742
743     ENDLOOP.
744     ENDIF.
745     DESCRIBE TABLE add_actor_tab LINES counter.
746     IF counter > 0.
747         * Acting positions found, replace original actor
748         acting = 'X'.
749         CLEAR actor_tab. REFRESH actor_tab.
750         actor_tab[] = add_actor_tab[].
751     ENDIF.
752
753     ENDFORM.                                " check_acting_manager
754     *&-----*
755     *&      Form  get_employee_number
756     *&-----*
757     *      text
758     *-----*
759     *      -->P_PERNR text
760     *      <--P_GV_CURR_PERNR text
761     *-----*
762     FORM get_employee_number USING p_username
763                                CHANGING p_pernr
764                                p_message.
765
766
767     CALL FUNCTION 'HRCA_GETEMPLOYEE_DATA_FROMUSER'
768     EXPORTING
769         username                = p_username
770     IMPORTING
771         employeenumber          = p_pernr
772     EXCEPTIONS
773         user_not_found          = 1
774         countrygrouping_not_found = 2
775         infity_not_found        = 3
776         call_others             = 4
777         OTHERS                  = 5.
778
779     IF sy-subrc NE 0.
780         CONCATENATE 'Error retrieving personnel number for user' p_username
781             INTO p_message SEPARATED BY space.
782     ENDIF.
783
784     ENDFORM.                                " get_employee_number
785     *&-----*
786     *&      Form  get_position
787     *&-----*
788     *      text
789     *-----*
790     *      -->P_GV_CURR_PERNR text
791     *      <--P_GV_CURR_PLANS text

```

```

792 *-----*
793 FORM get_position USING      p_pernr
794                             p_begda
795                             p_endda
796                             CHANGING p_plans
797                             p_message.
798
799 DATA: lt_1001 LIKE p1001 OCCURS 10 WITH HEADER LINE.
800
801 CLEAR p_plans.
802
803 CALL FUNCTION 'RH_READ_INFITY_1001'
804     EXPORTING
805         authority          = 'DISP'
806         with_stru_auth     = ' '
807         plvar              = '01'
808         otype              = 'P'
809         objid              = p_pernr
810         istat              = '1'
811         subty              = 'B008'
812         begda              = p_begda
813         endda              = p_endda
814     TABLES
815         i1001              = lt_1001
816 *     OBJECTS              =
817     EXCEPTIONS
818         nothing_found      = 1
819         wrong_condition    = 2
820         wrong_parameters   = 3
821         OTHERS             = 4.
822 IF sy-subrc <> 0.
823     If sy-langu = 'S'.
824         CONCATENATE 'No se encontró posición para el número de personal' p
825 _pernr
826         INTO p_message SEPARATED BY space.
827     ELSE.
828         CONCATENATE 'No position found for personnel number' p_pernr
829         INTO p_message SEPARATED BY space.
830     ENDIF.
831     EXIT.
832 ENDIF.
833 READ TABLE lt_1001 INDEX 1.
834 IF sy-subrc NE 0.
835 *     CONCATENATE 'No position found for personnel number' p_pernr
836 *     INTO p_message SEPARATED BY space.
837     If sy-langu = 'S'.
838         CONCATENATE 'No se encontró posición para el número de personal' p
839 _pernr
840         INTO p_message SEPARATED BY space.
841     ELSE.
842         CONCATENATE 'No position found for personnel number' p_pernr
843         INTO p_message SEPARATED BY space.
844     ENDIF.
845     EXIT.
846 ENDIF.
847 p_plans = lt_1001-sobid.
848
849 ENDFORM.                                " get_position
850

```

```

851 *-----*
852 *      Form  get_users_with_mss_role
853 *-----*
854 *      text
855 *-----*
856 *  -->  p1      text
857 *  <--  p2      text
858 *-----*
859 FORM get_users_with_mss_role .
860
861 DATA: lt_agr_define LIKE agr_define OCCURS 100 WITH HEADER LINE,
862        lt_activity_group_users LIKE agr_users
863                                OCCURS 1000 WITH HEADER LINE.
864 RANGES: r_agr_name FOR agr_define-agr_name.
865
866 REFRESH gt_users.
867
868 r_agr_name-sign = 'I'.
869 r_agr_name-option = 'CP'.
870 r_agr_name-low = 'Z*MSS*'.
871 APPEND r_agr_name.
872
873 * Get all MSS role
874 SELECT * FROM agr_define INTO TABLE lt_agr_define
875        WHERE agr_name IN r_agr_name.
876 r_agr_name-sign = 'I'.
877 r_agr_name-option = 'CP'.
878 r_agr_name-low = 'Z*MSS*'.
879 APPEND r_agr_name.
880
881 * Get all MSS roles
882 SELECT * FROM agr_define INTO TABLE lt_agr_define
883        WHERE agr_name IN r_agr_name.
884
885 LOOP AT lt_agr_define.
886   REFRESH lt_activity_group_users.
887   CALL FUNCTION 'RSRA_USERS_OF_AGR_GET'
888     EXPORTING
889       i_agr_name          = lt_agr_define-agr_name
890       time_dependent      = 'X'
891     TABLES
892       activity_groups_users = lt_activity_group_users
893     EXCEPTIONS
894       no_user_available    = 1
895       OTHERS               = 2.
896   LOOP AT lt_activity_group_users
897     WHERE from_dat LE sy-datum
898     AND   to_dat   GE sy-datum.
899     gt_users-uname = lt_activity_group_users-uname.
900     COLLECT gt_users.
901   ENDLOOP.    " AT lt_activity_group_users.
902
903   ENDLOOP.    " at lt_agr_define
904 ENDFORM.      " get_users_with_mss_role
905 *-----*
906 *      Form  get_employee_for_mss_users
907 *-----*
908 *      text
909 *-----*
910 *  -->  p1      text
911 *  <--  p2      text

```

```

912 *-----*
913 FORM get_employee_for_mss_users .
914
915     LOOP AT gt_users.
916         CHECK gt_users-uname NE sy-uname.
917         SELECT SINGLE pernr
918             FROM pa0105
919             INTO gt_mss_employees-pernr
920             WHERE usrid EQ gt_users-uname
921                 AND subty EQ '0001'
922                 AND begda LE sy-datum
923                 AND endda GE sy-datum.
924     IF sy-subrc EQ 0.
925         COLLECT gt_mss_employees.
926     ENDIF.
927 ENDLOOP.
928
929 ENDFORM.                                " get_employee_for_mss_users
930 *&-----*
931 *&      Form  get_employees_from_name
932 *&-----*
933 *      text
934 *-----*
935 *  →  p1      text
936 *  ←  p2      text
937 *-----*
938 FORM get_employees_from_name TABLES p_act_tab STRUCTURE zppa013
939     USING p_vorna
940           p_nachn
941           p_rufnm
942           p_perid
943           p_name2
944           orgtxt2
945           butxt2
946           namel.
947
948 DATA: lt_possible_employees LIKE zppa013 OCCURS 1000 WITH HEADER LINE,
949       lv_vorna_ok,
950       lv_nachn_ok,
951       lv_rufnm_ok,
952       lv_perid_ok,
953       lv_name2_ok,
954       lv_orgtxt2_ok,
955       lv_butxt2_ok,
956       lv_namel_ok.
957
958
959
960
961 SELECT * FROM pa0002
962     INTO CORRESPONDING FIELDS OF TABLE lt_possible_employees
963     FOR ALL ENTRIES IN gt_mss_employees
964     WHERE pernr EQ gt_mss_employees-pernr
965        AND begda LE sy-datum
966        AND endda GE sy-datum.
967
968
969 LOOP AT lt_possible_employees.
970
971
972     SELECT SINGLE orgeh bukrs werks FROM pa0001

```

```

973         INTO (lt_possible_employees-orgeh, lt_possible_employees-bukrs
974 , lt_possible_employees-werks)
975         WHERE pernr EQ lt_possible_employees-pernr
976         AND begda LE sy-datum
977         AND endda GE sy-datum.
978
979
980 *move org text
981
982
983
984         SELECT SINGLE orgtx
985         FROM t527x
986         INTO lt_possible_employees-orgtxt2
987         WHERE sprsl EQ sy-langu
988         AND orgeh EQ lt_possible_employees-orgeh
989         AND endda GE sy-datum
990         AND begda LE sy-datum.
991
992         IF sy-subrc <> 0.
993
994         SELECT SINGLE orgtx
995         FROM t527x
996         INTO lt_possible_employees-orgtxt2
997         WHERE sprsl EQ 'EN'
998         AND orgeh EQ lt_possible_employees-orgeh
999         AND endda GE sy-datum
1000         AND begda LE sy-datum.
1001
1002         ENDIF.
1003
1004
1005
1006 *move bukr text
1007         SELECT SINGLE butxt
1008         FROM t001
1009         INTO lt_possible_employees-butxt2
1010         WHERE bukr EQ lt_possible_employees-bukrs.
1011
1012
1013 *move all Personnel Area text
1014
1015
1016         SELECT SINGLE name1
1017         FROM t500p
1018         INTO lt_possible_employees-name1
1019         WHERE persa EQ lt_possible_employees-werks.
1020
1021         MODIFY lt_possible_employees.
1022
1023
1024         PERFORM check_search_field USING lt_possible_employees-orgtxt2
1025                                         orgtxt2
1026                                         CHANGING lv_orgtxt2_ok.
1027
1028         PERFORM check_search_field USING lt_possible_employees-butxt2
1029                                         butxt2
1030                                         CHANGING lv_butxt2_ok.
1031
1032         PERFORM check_search_field USING lt_possible_employees-name1

```

```

1033                                     name1
1034                                CHANGING lv_name1_ok.
1035
1036    PERFORM check_search_field USING lt_possible_employees-vorna
1037                                     p_vorna
1038                                CHANGING lv_vorna_ok.
1039
1040    PERFORM check_search_field USING lt_possible_employees-nachn
1041                                     p_nachn
1042                                CHANGING lv_nachn_ok.
1043
1044    PERFORM check_search_field USING lt_possible_employees-rufnm
1045                                     p_rufnm
1046                                CHANGING lv_rufnm_ok.
1047
1048    PERFORM check_search_field USING lt_possible_employees-perid
1049                                     p_perid
1050                                CHANGING lv_perid_ok.
1051
1052    PERFORM check_search_field USING lt_possible_employees-name2
1053                                     p_name2
1054                                CHANGING lv_name2_ok.
1055
1056
1057    IF lv_vorna_ok    = 'Y'    AND
1058       lv_nachn_ok    = 'Y'    AND
1059       lv_rufnm_ok    = 'Y'    AND
1060       lv_perid_ok    = 'Y'    AND
1061       lv_name2_ok    = 'Y'    AND
1062       lv_orgtxt2_ok  = 'Y'    AND
1063       lv_butxt2_ok   = 'Y'    AND
1064       lv_name1_ok    = 'Y'.
1065
1066    APPEND lt_possible_employees TO p_act_tab.
1067  ENDIF.
1068  ENDLOOP. " AT lt_possible_employees.
1069
1070  ENDFORM.                                " get_employees_from_name
1071  *&-----*
1072  *&      Form  check_search_field
1073  *&-----*
1074  *      text
1075  *-----*
1076  *      -->P LT_POSSIBLE_EMPLOYEES_VORNA  text
1077  *      -->P P_VORNA  text
1078  *      <--P LV_VORNA_OK  text
1079  *-----*
1080  FORM check_search_field  USING      p_field
1081                                     p_search_string
1082                                CHANGING p_ok_flag.
1083  DATA: lv_upper_case_field(40).
1084
1085  CLEAR p_ok_flag.
1086  lv_upper_case_field = p_field.
1087  TRANSLATE lv_upper_case_field TO UPPER CASE.
1088  IF lv_upper_case_field CP p_search_string.
1089    p_ok_flag = 'Y'.
1090  ENDIF.
1091
1092  ENDFORM.                                " check_search_field
1093  *&-----*

```

```

1094 * &      Form  get_employee_details
1095 * &-----*
1096 *      text
1097 *-----*
1098 *  →  p1      text
1099 *  ←  p2      text
1100 *-----*
1101 FORM get_employee_details  TABLES p_act_tab STRUCTURE zppa013.
1102
1103 DATA: lv_index LIKE sy-tabix.
1104
1105 LOOP AT p_act_tab.
1106     lv_index = sy-tabix.
1107
1108 * Get email address
1109     SELECT SINGLE usrid_long
1110           FROM pa0105
1111           INTO p_act_tab-email
1112           WHERE pernr EQ p_act_tab-pernr
1113           AND   subty EQ '0006'
1114           AND   begda LE sy-datum
1115           AND   endda GE sy-datum.
1116
1117 * Get org data
1118     SELECT SINGLE * FROM pa0001
1119           WHERE pernr EQ p_act_tab-pernr
1120           AND   begda LE sy-datum
1121           AND   endda GE sy-datum.
1122
1123 * Get texts
1124     SELECT SINGLE plstx
1125           FROM t528t
1126           INTO p_act_tab-planstxt
1127           WHERE sprsl EQ sy-langu
1128           AND   otype EQ 'S'
1129           AND   plans EQ pa0001-plans
1130           AND   endda GE sy-datum
1131           AND   begda LE sy-datum.
1132
1133     SELECT SINGLE orgtx
1134           FROM t527x
1135           INTO p_act_tab-orgtxt
1136           WHERE sprsl EQ sy-langu
1137           AND   orgeh EQ pa0001-orgeh
1138           AND   endda GE sy-datum
1139           AND   begda LE sy-datum.
1140
1141 IF sy-subrc <> 0.
1142
1143     SELECT SINGLE orgtx
1144           FROM t527x
1145           INTO p_act_tab-orgtxt
1146           WHERE sprsl EQ 'EN'
1147           AND   orgeh EQ pa0001-orgeh
1148           AND   endda GE sy-datum
1149           AND   begda LE sy-datum.
1150 ENDIF.
1151
1152 SELECT SINGLE butxt
1153           FROM t001
1154           INTO p_act_tab-butxt

```

```

1155         WHERE bukrs EQ pa0001-bukrs.
1156
1157     MODIFY p_act_tab INDEX lv_index.
1158
1159     ENDLOOP. " at p_act_tab
1160
1161 ENDFORM. " get_employee_details
1162 *&-----*
1163 *&      Form  send_change_email
1164 *&-----*
1165 *      text
1166 *-----*
1167 *  →  p1      text
1168 *  ←  p2      text
1169 *-----*
1170 FORM send_email USING p_type
1171                      begda
1172                      endda.
1173     DATA : lv_pernr      TYPE pernr_d.
1174     DATA : lv_mngrpernr  TYPE pernr_d.
1175     DATA : lv_actingname TYPE pad_vorna.
1176     DATA : lt_p0105      TYPE TABLE OF p0105 INITIAL SIZE 1.
1177     DATA : ls_p0105      TYPE p0105.
1178     DATA : ls_mngrname   TYPE pa0002.
1179     DATA : lv_usrid      TYPE sy-uname.
1180     DATA : lv_email      TYPE p0105-usrid_long.
1181     DATA : lv_begda(10),
1182             lv_endda(10).
1183     DATA : lv_user_address TYPE addr3_val,
1184             lv_user_usr03  TYPE usr03.
1185     DATA : lt_int_adr6    TYPE TABLE OF adsmtp INITIAL SIZE 1.
1186     DATA : ls_int_adr6    TYPE adsmtp.
1187     DATA : lv_sender      TYPE string.
1188     DATA : lv_subject     TYPE string,
1189             lv_recipient   TYPE string,
1190             lv_mail_text   TYPE string.
1191     DATA : BEGIN OF lt_email_text OCCURS 0,
1192             line(255),
1193     END OF lt_email_text.
1194
1195     DATA : lt_holder_tab TYPE TABLE OF swfactor INITIAL SIZE 1.
1196     DATA : ls_holder_tab TYPE swfactor.
1197
1198     DATA : lv_name TYPE thead-tdname. "DS1K954577
1199     DATA : lt_lines TYPE TABLE OF tline, "DS1K954577
1200             ls_line TYPE tline, "DS1K954577
1201             ls_header TYPE thead. "DS1K954577
1202     DATA : lv_text(255). "DS1K954577
1203     DATA : lv_langu TYPE sy-langu. "DS1K954577
1204
1205     CLEAR : lv_pernr, lt_p0105[], lv_email.
1206 * MM1 >>>> Send a mail to the acting person informing him/her of
1207 *           the assignment.
1208
1209 * Get the user assigned to the position
1210 CALL FUNCTION 'RH_STRUC_GET'
1211     EXPORTING
1212         act_otype      = 'S'
1213         act_objid      = gv_act_plans
1214         act_wegid      = 'A008'
1215         act_plvar      = ' '

```



```

1216      act_begda      = begda
1217      act_endda      = begda
1218      act_tdepth     = 2          " no recursion!
1219      authority_check = ' '
1220      TABLES
1221          result_tab      = lt_holder_tab
1222      EXCEPTIONS
1223          no_plvar_found  = 01
1224          no_entry_found  = 02.
1225
1226      LOOP AT lt_holder_tab INTO ls_holder_tab WHERE otype = 'P'.
1227          lv_pernr = ls_holder_tab-objid.
1228      ENDLOOP.
1229
1230      IF lv_pernr IS INITIAL.
1231          RETURN.
1232      ENDIF.
1233
1234      * Get the communication data
1235      CALL FUNCTION 'HR_READ_INFOTYPE'
1236          EXPORTING
1237              pernr      = lv_pernr
1238              infity     = '0105'
1239      *      begda      = begda
1240      *      endda     = begda
1241      TABLES
1242          infity_tab     = lt_p0105
1243      EXCEPTIONS
1244          infity_not_found = 1
1245          OTHERS         = 2.
1246
1247
1248      LOOP AT lt_p0105 INTO ls_p0105 WHERE subty = '0001'.
1249          lv_usrid = ls_p0105-usrid.
1250      ENDLOOP.
1251
1252      LOOP AT lt_p0105 INTO ls_p0105 WHERE subty = '0006'.
1253          lv_email = ls_p0105-usrid_long.
1254      ENDLOOP.
1255
1256      IF lv_email IS INITIAL.
1257      * Attempt to get the address from user master
1258          CALL FUNCTION 'SUSR_USER_ADDRESS_READ'
1259              EXPORTING
1260                  user_name      = lv_usrid
1261              IMPORTING
1262                  user_address    = lv_user_address
1263                  user_usr03     = lv_user_usr03
1264              EXCEPTIONS
1265                  user_address_not_found = 1
1266                  OTHERS              = 2.
1267
1268          REFRESH lt_int_adr6.
1269
1270          CALL FUNCTION 'ADDR_PERS_COMP_COMM_GET'
1271              EXPORTING
1272                  address_number    = lv_user_address-addrnumber
1273                  person_number     = lv_user_address-persnumber
1274                  table_type        = 'ADSMTP'
1275              TABLES
1276                  comm_table        = lt_int_adr6

```

```

1277      EXCEPTIONS
1278          parameter_error      = 1
1279          address_not_exist    = 2
1280          person_not_exist     = 3
1281          internal_error       = 4
1282          OTHERS                = 5.
1283
1284      LOOP AT lt_int_adr6 INTO ls_int_adr6 WHERE flgdefault = 'X' AND
1285              flg_nouse = ''.
1286          TRANSLATE ls_int_adr6-smtp_addr TO UPPER CASE.
1287          MODIFY lt_int_adr6 FROM ls_int_adr6 INDEX sy-tabix.
1288      ENDLOOP.
1289
1290      IF ls_int_adr6-smtp_addr IS INITIAL.
1291          RETURN.
1292      ELSE.
1293          lv_email = ls_int_adr6-smtp_addr.
1294      ENDIF.
1295
1296  ENDIF.
1297
1298  CLEAR : lv_actingname, lv_sender, ls_mngrname.
1299  *   Get the Acting Person's Name
1300  SELECT SINGLE vorna FROM pa0002 INTO lv_actingname
1301      WHERE pernr = lv_pernr
1302            AND begda <= begda
1303            AND endda >= begda.
1304
1305  *   Get the Person who is assigning acting name.
1306  REFRESH : lt_holder_tab[].
1307
1308  CALL FUNCTION 'RH_STRUC_GET'
1309      EXPORTING
1310          act_otype      = 'S'
1311          act_objid      = gv_curr_plans
1312          act_wegid      = 'A008'
1313          act_plvar      = ' '
1314          act_begda      = begda
1315          act_endda      = begda
1316          act_tdepth     = 2           " no recursion!
1317          authority_check = ' '
1318      TABLES
1319          result_tab     = lt_holder_tab
1320      EXCEPTIONS
1321          no_plvar_found = 01
1322          no_entry_found = 02.
1323
1324  LOOP AT lt_holder_tab INTO ls_holder_tab WHERE otype = 'P'.
1325      lv_mngrpernr = ls_holder_tab-objid.
1326  ENDLOOP.
1327
1328
1329  SELECT SINGLE * FROM pa0002 INTO ls_mngrname
1330      WHERE pernr = lv_mngrpernr
1331            AND begda <= begda
1332            AND endda >= begda.
1333
1334  lv_sender = 'WF-BATCH@ANGLOAMERICAN.CO.ZA'.
1335
1336  * Format the dates
1337  CONCATENATE begda+6(2) begda+4(2) begda(4) INTO lv_begda

```

```

1338             SEPARATED BY ' '.
1339
1340     CONCATENATE endda+6(2) endda+4(2) endda(4) INTO lv_endda
1341             SEPARATED BY ' '.
1342
1343 * DS1K954577 BEGIN
1344 ** Subject
1345 *   lv_subject = 'Acting Approver'.
1346 *
1347 * E-mail body
1348 **   Dear &ACTINGAPPROVERNAME&,
1349 *   CONCATENATE lv_actingname ',' INTO lt_email_text-line.
1350 *   CONCATENATE 'Dear' lt_email_text-line INTO lt_email_text-line
1351 *           SEPARATED BY space.
1352 *   APPEND lt_email_text.
1353 *   CLEAR lt_email_text-line.
1354 *   APPEND lt_email_text.
1355 *
1356 ** You have been created as an acting approver by &MANAGERNAME&
1357 ** for the period of &START to END DATE&.
1358 *   IF p_type = 'Create'.
1359 *       CONCATENATE lv_endda '.' INTO lt_email_text-line.
1360 *       CONCATENATE 'You have been created as an acting approver by'
1361 *               ls_mngrname-vorna ls_mngrname-nachn 'for the period of'
1362 *               lv_begda 'to' lt_email_text-line INTO lt_email_text-li
ne
1363 *               SEPARATED BY space.
1364 *
1365 *   ELSEIF p_type = 'Change'.
1366 *
1367 *       CONCATENATE 'Please note that' ls_mngrname-vorna ls_mngrname-nachn
1368 *               'has updated the acting approver record for you.'
1369 *       INTO lt_email_text-line SEPARATED BY space.
1370 *
1371 *   ELSEIF p_type = 'Delete'.
1372 *
1373 *       CONCATENATE 'Please note that your acting approver record has been
deleted by'
1374 *               ls_mngrname-vorna ls_mngrname-nachn
1375 *       INTO lt_email_text-line SEPARATED BY space.
1376 *
1377 *   ENDIF.
1378 *
1379 *   APPEND lt_email_text.
1380 *   CLEAR lt_email_text-line.
1381 *   APPEND lt_email_text.
1382 *   CLEAR lt_email_text-line.
1383 *
1384 ** Please logon to the Manager Self Service page at http://fusion.anglo.
local
1385 ** and follow the link to the Work Overview page in order to process the
leave request.
1386 *   CONCATENATE text-001 text-002 INTO lt_email_text-line SEPARATED BY sp
ace.
1387 *   APPEND lt_email_text.
1388 *   CLEAR lt_email_text-line.
1389
1390 gv_actingname = lv_actingname.
1391 gs_mngrname = ls_mngrname.
1392 gv_begda = lv_begda.
1393 gv_endda = lv_endda.

```

```

1394
1395 * Set text name and subject
1396 CASE p_type.
1397     WHEN 'Create'.
1398         lv_name = 'ZACTING CREATE'.
1399         lv_subject = text-003.
1400     WHEN 'Change'.
1401         lv_name = 'ZACTING CHANGE'.
1402         lv_subject = text-004.
1403     WHEN 'Delete'.
1404         lv_name = 'ZACTING DELETE'.
1405         lv_subject = text-005.
1406 ENDCASE.
1407
1408 * Read Text
1409 lv_langu = sy-langu.
1410 CALL FUNCTION 'READ_TEXT'
1411     EXPORTING
1412         id                = 'ST'
1413         language          = lv_langu
1414         name              = lv_name
1415         object            = 'TEXT'
1416     IMPORTING
1417         header            = ls_header
1418     TABLES
1419         lines             = lt_lines
1420     EXCEPTIONS
1421         id                = 1
1422         language          = 2
1423         name              = 3
1424         not_found         = 4
1425         object            = 5
1426         reference_check   = 6
1427         wrong_access_to_archive = 7
1428         OTHERS            = 8.
1429
1430 * Replace variables with content
1431 CALL FUNCTION 'TEXT_SYMBOL_REPLACE'
1432     EXPORTING
1433         header            = ls_header
1434         program           = sy-repid
1435         replace_text      = 'X'
1436     TABLES
1437         lines             = lt_lines.
1438
1439 * Build table lt_email_text
1440 READ TABLE lt_lines INTO ls_line INDEX 1.
1441
1442 lv_text = ls_line-tdline.
1443
1444 LOOP AT lt_lines INTO ls_line FROM 2.
1445     IF ls_line-tdformat = 'T'.
1446         CONCATENATE lv_text ls_line-tdline
1447             INTO lv_text SEPARATED BY space.
1448     ELSE.
1449         APPEND lv_text TO lt_email_text.
1450         lv_text = ls_line-tdline.
1451     ENDIF.
1452 ENDLOOP.
1453
1454 APPEND lv_text TO lt_email_text.

```

```
1455 * DS1K954577 END
1456
1457     lv_recipient = lv_email.
1458
1459     CALL FUNCTION 'HR_FBN_GENERATE_SEND_EMAIL'
1460         EXPORTING
1461             subject          = lv_subject
1462             sender           = lv_sender
1463             recipient        = lv_recipient
1464             flag_commit      = 'X'
1465             flag_send_immediately = ' '
1466         TABLES
1467             email_text       = lt_email_text
1468         EXCEPTIONS
1469             failed           = 1
1470             OTHERS           = 2.
1471
1472     ENDFORM.                                " send_change_email
```

Subroutines:		Line	Page
ABORT.....	LZSWXAF01	23	13
	LZSWXAF01	67	14
	LZSWXAF01	85	14
	LZSWXAF01	186	16
CHECK_ACTING_MANAGER.....	LZSWXAF01	587	22
CHECK_SEARCH_FIELD.....	LZSWXAF01	1024	29
	LZSWXAF01	1028	29
	LZSWXAF01	1032	29
	LZSWXAF01	1036	30
	LZSWXAF01	1040	30
	LZSWXAF01	1044	30
	LZSWXAF01	1048	30
	LZSWXAF01	1052	30
	LZSWXAF01	1080	30
CREATION INITIALIZE.....	LZSWXAF01	553	22
DYNPRO_010_INIT.....	LZSWXAF01	398	19
DYNPRO_050_INIT.....	LZSWXAF01	460	20
DYNPRO_051_INIT.....	LZSWXAF01	315	18
EXIT.....	LZSWXAF01	124	15
	LZSWXAF01	149	15
	LZSWXAF01	206	16
FORM_APPROVE.....	LZSWXAF01	27	13
	LZSWXAF01	221	16
FORM_CHECK_APPROVED.....	LZSWXAF01	146	15
	LZSWXAF01	161	15
FORM_CHECK_SAVED.....	LZSWXAF01	46	13
	LZSWXAF01	142	15
	LZSWXAF01	144	15
FORM_EXIT.....	LZSWXAF01	19	13
	LZSWXAF01	21	13
	LZSWXAF01	138	15
FORM_LAYOUT.....	LZSWXAF01	228	16
	LZSWXAF01	249	17
	LZSWXAF01	263	17
FORM_PROCESS.....	LZSWXAF01	12	13
FORM_REJECT.....	LZSWXAF01	29	13
	LZSWXAF01	242	16
FORM_SAVE_AND_EXIT.....	LZSWXAF01	25	13
	LZSWXAF01	65	14
	LZSWXAF01	101	14
	LZSWXAF01	168	15
GET_EMPLOYEE_DETAILS.....	LZSWXAF01	1101	31
GET_EMPLOYEES FROM NAME.....	LZSWXAF01	938	28
GET_EMPLOYEE FOR MSS_USERS....	LZSWXAF01	913	28
GET_EMPLOYEE_NUMBER.....	LZSWXAF01	762	25
GET_NUMBER.....	LZSWXAF01	513	21
GET_POSITION.....	LZSWXAF01	793	26
GET_USERS WITH MSS_ROLE.....	LZSWXAF01	859	27
INITIALIZE.....	LZSWXAF01	576	22
SCREEN_011.....	LZSWXAF01	382	19
SEND_EMAIL.....	LZSWXAF01	1170	32

Status:		Line	Page
FORMSTAT.....	LZSWXAF01	449	20

Messages:

		Line	Page
WZ002.....	LZSWXAF01	117	14
WZ003.....	LZSWXAF01	120	14
WZ006.....	LZSWXAF01	229	16
WZ008.....	LZSWXAF01	250	17
WZ800.....	LZSWXAF01	71	14
	LZSWXAF01	112	14
	LZSWXAF01	192	16
	LZSWXAF01	301	17
	LZSWXAF01	420	19
	LZSWXAF01	447	20
	LZSWXAF01	483	20

External calls

Function modules:

Function modules:	Line	Page
ADDR_PERS_COMP_COMM_GET.....	LZSWXAF01	1270 33
HRCA_GETEMPLOYEE_DATA_FROMUSER.	LZSWXAF01	767 25
HR_FBN_GENERATE_SEND_EMAIL....	LZSWXAF01	1459 37
HR_READ_INFOTYPE.....	LZSWXAF01	1235 33
NUMBER_GET_NEXT.....	LZSWXAF01	515 21
POPUP_TO_CONFIRM_STEP.....	LZSWXAF01	54 13
	LZSWXAF01	175 15
READ_TEXT.....	LZSWXAF01	1410 36
RH_READ_INFNTY_1001.....	LZSWXAF01	803 26
RH_STRUC_GET.....	LZSWXAF01	627 23
	LZSWXAF01	646 23
	LZSWXAF01	664 23
	LZSWXAF01	683 24
	LZSWXAF01	1210 32
	LZSWXAF01	1308 34
RSRA_USERS_OF_AGR_GET.....	LZSWXAF01	887 27
SUSR_USER_ADDRESS_READ.....	LZSWXAF01	1258 33
SWL_AGENTS_NAME_DETERMIN.....	LZSWXAF01	560 22
SWU_NUMBER_RANGE_CREATE.....	LZSWXAF01	531 21
TEXT_SYMBOL_REPLACE.....	LZSWXAF01	1431 36
ZSWX_GET_DIRECT_MANAGER.....	LZSWXAF01	727 24

DATABASE OPERATIONS

SELECT:		Line	Page
AGR_DEFINE.....	LZSWXAF01	874	27
	LZSWXAF01	882	27
PA0001.....	LZSWXAF01	972	28
	LZSWXAF01	1118	31
PA0002.....	LZSWXAF01	961	28
	LZSWXAF01	1300	34
	LZSWXAF01	1329	34
PA0105.....	LZSWXAF01	715	24
	LZSWXAF01	918	28
	LZSWXAF01	1110	31
T001.....	LZSWXAF01	1008	29
	LZSWXAF01	1153	31
T500P.....	LZSWXAF01	1017	29
T527X.....	LZSWXAF01	985	29
	LZSWXAF01	995	29
	LZSWXAF01	1134	31
	LZSWXAF01	1144	31
T528T.....	LZSWXAF01	1125	31

UPDATE:		Line	Page
SWXFORMABS.....	LZSWXAF01	108	14
	LZSWXAF01	110	14

MODIFY:		Line	Page
LT_INT_ADR6.....	LZSWXAF01	1287	34

INTERNAL TABLE OPERATIONS

READ TABLE:		Line	Page
LT_1001.....	LZSWXAF01	833	26
LT_LINES.....	LZSWXAF01	1440	36

MODIFY:		Line	Page
LT_POSSIBLE_EMPLOYEES.....	LZSWXAF01	1021	29
P_ACT_TAB.....	LZSWXAF01	1157	32

DELETE:		Line	Page
ADD_ACTOR_TAB.....	LZSWXAF01	712	24

```

1  *-----*
2  ***INCLUDE LZSWXAF02 .
3  *-----*
4  *&-----*
5  *&      Form  GET_EE_POSITION
6  *&-----*
7  *      text
8  *-----*
9  *      -->P_ORG_OBJECT  text
10 *      <--P_LV_POSTION  text
11 *-----*
12 FORM get_ee_data USING      p_org_object STRUCTURE swhactor
13                             p_datum
14                             CHANGING p_pernr
15                             p_position
16                             p_acting_vac_flag
17                             p_vac_reports_to
18                             p_bukrs
19                             p_persg.
20
21 DATA BEGIN OF position_tab OCCURS 0.
22 INCLUDE STRUCTURE swhactor.
23 DATA END OF position_tab.
24
25 DATA BEGIN OF user_or_person_tab OCCURS 0.
26 INCLUDE STRUCTURE swhactor.
27 DATA END OF user_or_person_tab.
28
29 DATA: ls_zppa000 TYPE  zppa000.
30
31 DATA: lv_sobid  LIKE hrp1001-sobid,
32        lv_sobid1 LIKE hrp1001-sobid.
33
34 CASE p_org_object-otype.
35 *-----*
36 * Object Type = USER
37 *-----*
38 WHEN org_objtype_user.
39 APPEND p_org_object TO user_or_person_tab.
40 * Get person
41 SELECT SINGLE pernr
42 FROM pa0105
43 INTO p_pernr
44 WHERE sprps EQ space
45 AND subty = '0001'
46 AND begda LE p_datum
47 AND endda GE p_datum
48 AND usrid = p_org_object-objid.
49
50 IF sy-subrc EQ 0 AND p_pernr IS NOT INITIAL.
51
52 CALL FUNCTION 'Z_GET_COMPANY_CODE_GROUPING'
53 EXPORTING
54     pernr      = p_pernr
55     datum      = p_datum
56 IMPORTING
57     oszppa000  = ls_zppa000
58 EXCEPTIONS
59     no_company_code_group_found = 1
60     OTHERS                    = 2.
61

```

```

62         p_acting_vac_flag = ls_zppa000-zzactvac.
63         p_vac_reports_to   = ls_zppa000-zzvacreportsto.
64
65         user_or_person_tab-otype = 'P'.
66         user_or_person_tab-objid = p_pernr.
67         APPEND user_or_person_tab.
68
69     ENDIF.
70
71     *-----*
72     * Object Type = PERSON
73     *-----*
74     WHEN org_objtype_person.
75         p_pernr = p_org_object-objid.
76         APPEND p_org_object TO user_or_person_tab.
77
78     *-----*
79     * Object Type = POSITION
80     *-----*
81
82     WHEN org_objtype_position.
83         "APPEND p_org_object TO position_tab.
84         p_position = p_org_object-objid.
85         SELECT SINGLE sobid INTO lv_sobid
86         FROM hrp1001
87         WHERE plvar = '01'
88             AND otype = 'S'
89             AND objid = p_position
90             AND rsign = 'A'
91             AND relat = '008'
92             AND subty = 'A008'
93             AND begda LE p_datum
94             AND endda GE p_datum
95             AND sclas = 'P'.
96         IF sy-subrc EQ 0.
97             p_pernr = lv_sobid(8).
98         ENDIF.
99         RETURN.
100
101     *-----*
102     * Object Type = ORGUNIT
103     *-----*
104     WHEN org_objtype_orgunit.
105         "APPEND p_org_object TO orgunit_tab.
106     *     MESSAGE e010 WITH space space space space
107     *     RAISING nobody_found.
108     * Get chief position of the orgunit
109         SELECT SINGLE sobid INTO lv_sobid
110         FROM hrp1001
111         WHERE plvar = '01'
112             AND otype = 'O'
113             AND objid = p_org_object-objid
114             AND rsign = 'B'
115             AND relat = '012'
116             AND subty = 'B012'
117             AND begda LE p_datum
118             AND endda GE p_datum
119             AND sclas = 'S'.
120         IF sy-subrc EQ 0.
121             p_position = lv_sobid(8).
122         RETURN.

```

```

123         ELSE.
124             MESSAGE e010 WITH space space space space
125                 RAISING nobody_found.
126             RETURN.
127         ENDIF.
128 *-----*
129 * Object Type = OTHER
130 *-----*
131     WHEN OTHERS.
132         MESSAGE e010 WITH space space space space
133             RAISING nobody_found.
134     ENDCASE.
135
136 *****
137 *** 1. Get the position of a user or person ***
138 *** user/person — wf_posit —> position ***
139 *** The user/person under consideration is in ***
140 *** user_or_person_tab. ***
141 *****
142
143     LOOP AT user_or_person_tab.         " user_or_person_tab has 0 or 1 entries
144
145 * Get the position of the user/person.
146     CALL FUNCTION 'RH_STRUC_GET'
147         EXPORTING
148             act_otype          = user_or_person_tab-otype
149             act_objid          = user_or_person_tab-objid
150             act_wegid          = 'WF_POSIT'
151             act_plvar          = ' '
152             act_begda          = p_datum
153             act_endda          = p_datum
154             act_tdepth         = 0
155             authority_check    = ' '
156 * importing
157 *     act_plvar          =
158     TABLES
159         result_tab          = position_tab
160 *     result_objec       = dummy_result_objec
161 *     result_struc       = dummy_result_struc
162     EXCEPTIONS
163         no_plvar_found      = 01
164         no_entry_found      = 02.
165
166     IF sy-subrc NE 0.
167         MESSAGE e010 WITH space space space space
168             RAISING nobody_found.
169     ENDIF.
170     LOOP AT position_tab WHERE otype = 'S'.
171         p_position = position_tab-objid.
172     ENDLOOP.
173 ENDLOOP.
174
175 * Get employee company and employee group
176 CLEAR: p_bukrs, p_persg.
177 IF p_pernr IS NOT INITIAL.
178     SELECT SINGLE bukrs persg
179     INTO (p_bukrs,p_persg)
180     FROM pa0001
181     WHERE pernr = p_pernr
182         AND sprps EQ space
183         AND begda LE gv_effdate

```

```

184         AND endda GE gv_effdate.
185     ENDIF.
186
187 ENDFORM.                                "get_ee_data
188
189 *-----*
190 *      Form  get_reports_to_position
191 *-----*
192 *      text
193 *-----*
194 *      —>P_POSITION          text
195 *      —>P_REPORTS_TO_POSITION text
196 *-----*
197 FORM get_reports_to_position USING    p_position p_datum
198                                     CHANGING p_reports_to_position.
199 DATA BEGIN OF lt_reports_to_tab OCCURS 0.
200 INCLUDE STRUCTURE swfactor.
201 DATA END OF lt_reports_to_tab.
202
203 CLEAR p_reports_to_position.
204 CALL FUNCTION 'RH_STRUC_GET'
205     EXPORTING
206         act_otype          = 'S'
207         act_objid          = p_position
208         act_wegid          = 'A002'
209         act_plvar          = ' '
210         act_begda          = p_datum
211         act_endda          = p_datum
212         act_tdepth         = 2          " no recursion!
213         authority_check    = ' '
214     TABLES
215         result_tab         = lt_reports_to_tab
216     EXCEPTIONS
217         no_plvar_found     = 01
218         no_entry_found     = 02.
219
220 ** Positions with A002 found - check for chiefs
221 * LOOP AT lt_reports_to_tab.
222 ** Check if the position is a chief position
223 *     CALL FUNCTION 'RH_STRUC_GET'
224 *     EXPORTING
225 *         act_otype          = lt_reports_to_tab-otype
226 *         act_objid          = lt_reports_to_tab-objid
227 *         act_wegid          = 'A012'
228 *         act_plvar          = ' '
229 *         act_begda          = sy-datum
230 *         act_endda          = sy-datum
231 *         act_tdepth         = 2          " no recursion!
232 *         authority_check    = ' '
233 *     EXCEPTIONS
234 *         no_plvar_found     = 01
235 *         no_entry_found     = 02.
236 *
237 *     IF sy-subrc = 0.
238 ** Chief position - check for holder
239 *         p_reports_to_position = lt_reports_to_tab-objid.
240 *         EXIT.
241 *     ENDIF.
242 * ENDLOOP.          " AT reports_to_tab
243
244 READ TABLE lt_reports_to_tab INDEX 1.

```

```

245
246 IF sy-subrc = 0.
247     p_reports_to_position = lt_reports_to_tab-objid.
248
249 ENDIF.
250
251 ENDFORM.                                " GET_REPORTS_TO_POSITION
252 *&-----*
253 *&      Form  GET_HOLDER
254 *&-----*
255 *      text
256 *-----*
257 *      -->P_ACTOR_TAB  text
258 *      -->P_LV_REPORTS_TO_POSITION  text
259 *-----*
260 FORM get_holder  TABLES  p_holder_tab STRUCTURE swfactor
261                      USING  p_reports_to_position
262                          p_datum
263                          p_pernr.
264
265 DATA: lv_stat2 LIKE p0000-stat2.
266
267 DATA BEGIN OF lt_holder_tab OCCURS 0.
268 INCLUDE STRUCTURE swfactor.
269 DATA END OF lt_holder_tab.
270
271 CALL FUNCTION 'RH_STRUC_GET'
272     EXPORTING
273         act_otype           = 'S'
274         act_objid           = p_reports_to_position
275         act_wegid           = 'A008'
276         act_plvar           = ' '
277         act_begda           = p_datum
278         act_endda           = p_datum
279         act_tdepth          = 2          " no recursion!
280         authority_check     = ' '
281     TABLES
282         result_tab          = lt_holder_tab
283     EXCEPTIONS
284         no_plvar_found      = 01
285         no_entry_found      = 02.
286
287 LOOP AT lt_holder_tab WHERE otype = 'P'.
288 * RT17935 - Remove reports to oneself
289     IF lt_holder_tab-objid = p_pernr.
290         DELETE lt_holder_tab.
291     ELSE.
292 * RT17395 - Only consider active holders
293         PERFORM get_stat2 USING lt_holder_tab-objid
294                                p_datum
295                                CHANGING lv_stat2.
296         IF lv_stat2 NE '0'.
297             APPEND lt_holder_tab TO p_holder_tab.
298         ENDIF.
299     ENDIF.
300 ENDLOOP.
301
302 ENDFORM.                                " GET_HOLDER
303 *&-----*
304 *&      Form  GET_ORG_UNITS
305 *&-----*

```

```

306 *      text
307 *-----*
308 *      -->P_ORGUNIT_TAB  text
309 *      -->P_LV_POSITION  text
310 *-----*
311 FORM get_org_units  TABLES  p_orgunit_tab STRUCTURE swfactor
312                          USING  p_position
313                          p_datum.
314
315
316 CLEAR p_orgunit_tab. REFRESH p_orgunit_tab.
317 eval_path = 'S-O-O'.          " position->org-unit->org-unit
318 eval_depth = 0.
319 *  Get all orgs upwards in the hierarchy
320 CALL FUNCTION 'RH_STRUC_GET'
321   EXPORTING
322     act_otype      = 'S'
323     act_objid      = p_position
324     act_begda      = p_datum
325     act_endda      = p_datum
326     act_wegid      = eval_path
327     act_plvar      = '01'
328     act_tdepth     = eval_depth " recursion check!
329     authority_check = ' '
330   TABLES
331     result_tab     = p_orgunit_tab
332   EXCEPTIONS
333     no_plvar_found = 01
334     no_entry_found = 02.
335
336
337 ENDFORM.          " GET_ORG_UNITS
338 *&-----*
339 *&      Form  GET_CHIEF_POSITIONS
340 *&-----*
341 *      text
342 *-----*
343 *      -->P_ORGUNIT_TAB  text
344 *      -->P_CHIEF_TAB   text
345 *-----*
346 FORM get_chief_positions  TABLES  p_orgunit_tab STRUCTURE swfactor
347                                p_chief_tab STRUCTURE swfactor
348                                USING  p_datum.
349
350 DATA BEGIN OF lt_chief_tab OCCURS 0.
351 INCLUDE STRUCTURE swfactor.
352 DATA END OF lt_chief_tab.
353
354 REFRESH p_chief_tab.
355
356 * Check for the managers of the org-units found.
357 LOOP AT p_orgunit_tab.
358   CLEAR lt_chief_tab. REFRESH lt_chief_tab.
359   CALL FUNCTION 'RH_STRUC_GET'
360     EXPORTING
361       act_otype      = p_orgunit_tab-otype
362       act_objid      = p_orgunit_tab-objid
363       act_begda      = p_datum
364       act_endda      = p_datum
365       act_wegid      = 'B012'
366       act_plvar      = '01'

```



```

367         act_tdepth      = 2           " no recursion!
368         authority_check = ' '
369     TABLES
370         result_tab       = lt_chief_tab
371     EXCEPTIONS
372         no_plvar_found    = 01
373         no_entry_found    = 02.
374
375     IF gv_explain EQ 'X'.
376         LOOP AT lt_chief_tab.
377             PERFORM details_line TABLES gt_details_tab USING:
378                 p_orgunit_tab-objid 'Has Chief Position'
379                 'S' lt_chief_tab-objid.
380         ENDLOOP.
381     ENDIF.
382     APPEND LINES OF lt_chief_tab TO p_chief_tab.
383 ENDLOOP.
384
385 ENDFORM.                                " GET_CHIEF_POSITIONS
386 *&-----*
387 *&      Form  CHECK_ACTING_POSITION
388 *&-----*
389 *      text
390 *-----*
391 *      -->P_ACTOR_TAB    text
392 *      -->P_LV_REPORTS_TO_POSITION  text
393 *-----*
394 FORM check_acting_position TABLES p_actor_tab STRUCTURE swactor
395                                USING p_pernr
396                                p_reports_to_position
397                                p_datum
398                                CHANGING acting.
399
400
401
402 DATA BEGIN OF acting_tab OCCURS 0.
403 INCLUDE STRUCTURE swactor.
404 DATA END OF acting_tab.
405
406 DATA BEGIN OF add_actor_tab OCCURS 0.
407 INCLUDE STRUCTURE swactor.
408 DATA END OF add_actor_tab.
409
410 DATA: lv_usrid    LIKE pa0105-usrid,
411        lv_own_user.
412 DATA: lt_actor_tab    LIKE swactor OCCURS 10 WITH HEADER LINE,
413        lt_ac_container LIKE swcont OCCURS 10 WITH HEADER LINE.
414
415 lv_own_user = 'N'.
416 * check for acting relationship from the chief's position
417 CLEAR add_actor_tab. REFRESH add_actor_tab.
418
419 CLEAR acting_tab. REFRESH acting_tab.
420 CALL FUNCTION 'RH_STRUC_GET'
421     EXPORTING
422         act_otype      = 'S'
423         act_objid      = p_reports_to_position
424         act_begda      = p_datum
425         act_endda      = p_datum
426         act_wegid      = 'ZAZ15'
427         act_plvar      = '01'

```

```

428         act_tdepth      = 2           " no recursion!
429         authority_check = ' '
430     TABLES
431         result_tab       = acting_tab
432     EXCEPTIONS
433         no_plvar_found   = 01
434         no_entry_found   = 02.
435
436     IF sy-subrc NE 0. "Z15 not found
437 * Check for Z08 relationship (this is to person directly)
438     CLEAR acting_tab. REFRESH acting_tab.
439     CALL FUNCTION 'RH_STRUC_GET'
440         EXPORTING
441             act_otype      = 'S'
442             act_objid      = p_reports_to_position
443             act_begda      = p_datum
444             act_endda      = p_datum
445             act_wegid      = 'ZAZ08'
446             act_plvar      = '01'
447             act_tdepth     = 2           " no recursion!
448             authority_check = ' '
449     TABLES
450         result_tab       = add_actor_tab
451     EXCEPTIONS
452         no_plvar_found   = 01
453         no_entry_found   = 02.
454
455 * RT17935 - For Z08 persons, remove self reporting
456     IF sy-subrc EQ 0.
457         LOOP AT add_actor_tab
458             WHERE otype = 'P' AND objid = p_pernr.
459             DELETE add_actor_tab.
460         ENDLOOP.
461     ENDIF.
462
463 * RT17935 - Else check that acting positions not exceptions
464 ELSE. "Z15 relationships found
465
466     LOOP AT acting_tab WHERE otype = 'S'.
467         IF acting_tab-objid IN rng_ignpos.
468             gv_wfignore = 'X'.
469             DELETE acting_tab.
470         ENDIF.
471     ENDLOOP.
472     DESCRIBE TABLE acting_tab LINES counter.
473     IF counter > 0.
474 * Valid acting positions, so continue
475         CLEAR gv_wfignore.
476     ELSE.
477         RETURN.
478     ENDIF.
479
480 * RT17935 - Check holders of acting positions
481     LOOP AT acting_tab WHERE otype = 'S'.
482         PERFORM get_holder TABLES add_actor_tab
483             USING acting_tab-objid
484                 p_datum
485                 p_pernr.
486     ENDLOOP.
487 ENDIF.
488

```

```

489
490 * RT17935 - remove holder check since this is done above
491 * LOOP AT acting_tab where otype = 'S'.
492 *
493 ** Get holders
494 * CLEAR add_actor_tab. REFRESH add_actor_tab.
495 * CALL FUNCTION 'RH_STRUC_GET'
496 * EXPORTING
497 * act_otype = acting_tab-otype
498 * act_objid = acting_tab-objid
499 * act_wegid = 'A008'
500 * act_plvar = ' '
501 * act_begda = p_datum
502 * act_endda = p_datum
503 * act_tdepth = 2 " no recursion!
504 * authority_check = ' '
505 * TABLES
506 * result_tab = add_actor_tab
507 * EXCEPTIONS
508 * no_plvar_found = 01
509 * no_entry_found = 02.
510 *
511 * ENDLOOP. " AT acting_tab
512
513
514 * ADD_ACTOR_TAB now contains the holders of the acting positions!
515
516 * Remove the user themselves
517 * RT17935 - already done above
518 * clear lv_own_user.
519 * LOOP AT add_actor_tab
520 * WHERE otype = 'P' and objid = p_pernr.
521 * delete add_actor_tab.
522 * ENDLOOP.
523
524 DESCRIBE TABLE add_actor_tab LINES counter.
525 IF counter > 0.
526 * Acting persons found, replace original actor tab
527 acting = 'X'.
528 CLEAR p_actor_tab. REFRESH p_actor_tab.
529 p_actor_tab[] = add_actor_tab[].
530 ENDIF.
531
532 ENDFORM. " CHECK_ACTING_POSITION
533 *&-----*
534 *& Form GET_VALIDATE_REPORTS_TO
535 *&-----*
536
537 FORM get_validate_reports_to TABLES actor_tab TYPE tswhactor
538 USING p_position
539 p_acting_vac_flag
540 p_pernr
541 p_datum
542 CHANGING p_reports_to_position
543 acting TYPE boole.
544
545 *&-----*
546 * Check if the position reports to another position.
547 PERFORM get_reports_to_position USING p_position p_datum
548 CHANGING p_reports_to_position.
549

```

```

550 * RT17395 If "A002" position not exists then return
551 IF p_reports_to_position IS INITIAL.
552 RETURN.
553 ENDIF.
554
555 * RT17395 If "A002" position is in the ignore list
556 * and acting not to be considered ==> return
557 IF p_reports_to_position IN rng_ignpos
558 AND gv_ignact IS NOT INITIAL.
559 gv_wfignore = 'X'.
560 RETURN.
561 ENDIF.
562
563 *-----*
564
565 IF ( p_acting_vac_flag = '1' OR
566 p_acting_vac_flag = '2' )
567 AND gv_ignact IS INITIAL.
568 * Acting for a vacant position allowed - check for acting before
569 * checking for holder.
570 PERFORM check_acting_position TABLES actor_tab
571 USING p_pernr
572 p_reports_to_position
573 p_datum
574 CHANGING acting. "position IDs
575
576 * Check whether we have finally found some manager.
577
578 * RT17395 - Exception position handling done in form above
579 * LOOP AT actor_tab.
580 * IF actor_tab-objid = '05000473' OR
581 * actor_tab-objid = '05000644'.
582 * DELETE actor_tab INDEX sy-tabix.
583 * ENDIF.
584 * ENDLOOP.
585
586 * RT17935 - check if exception positions were excluded
587 IF gv_wfignore IS NOT INITIAL.
588 RETURN.
589 ENDIF.
590
591 ELSE.
592
593 * Find a holder of the position
594 * RT17395 - this form also checks that holder is active and not self
595 PERFORM get_holder TABLES actor_tab
596 USING p_reports_to_position
597 p_datum
598 p_pernr.
599 ENDIF. "of acting check
600
601 *-----*
602 * Check whether we have finally found some manager.
603 DESCRIBE TABLE actor_tab LINES counter.
604 IF counter > 0. " manager found
605 IF gv_ignact IS INITIAL.
606 PERFORM check_acting_manager TABLES actor_tab
607 USING p_pernr
608 p_datum
609 CHANGING acting.
610 ELSE.

```

```

611         CLEAR acting.
612     ENDIF.
613 ENDIF.
614
615 * RT17935 - removed since this is done elsewhere
616 ** Check whether we have finally found some manager.
617 * LOOP AT actor_tab.
618 *     IF actor_tab-objid = '05000473' OR "Rene Medori
619 *         actor_tab-objid = '05000644' OR "Cynthia Carrol
620 *         actor_tab-objid = '05001534'. "Mark Cutifani
621 *         DELETE actor_tab INDEX sy-tabix.
622 *     ENDIF.
623 * ENDLOOP.
624
625 * DESCRIBE TABLE actor_tab LINES counter.
626 * IF counter > 0.
627 *     EXIT.
628 * ENDIF.
629
630 *-----*
631 ENDFORM.                                " GET_VALIDATE_REPORTS_TO
632 *&-----*
633 *&         Form GET_STAT2
634 *&-----*
635 FORM get_stat2 USING p_pernr
636                   p_datum
637                   CHANGING p_stat2.
638
639 SELECT SINGLE stat2 INTO p_stat2
640 FROM pa0000
641 WHERE pernr = p_pernr
642        AND sprps = space
643        AND begda LE p_datum
644        AND endda GE p_datum.
645
646 ENDFORM.                                " GET_STAT2
647 *&-----*
648 *&         Form CHECK_POSITION_ACTING
649 *&-----*
650 FORM check_position_acting USING p_pernr
651                               p_datum
652                               CHANGING p_position
653                               p_acting.
654
655 DATA: lt_acting_pos TYPE swactor OCCURS 10 WITH HEADER LINE.
656 DATA: lt_acting_per TYPE swactor OCCURS 10 WITH HEADER LINE.
657
658 DATA: lv_position LIKE swactor-objid.
659
660 REFRESH: lt_acting_pos, lt_acting_per.
661
662
663 CALL FUNCTION 'RH_STRUC_GET'
664     EXPORTING
665         act_otype           = 'S'
666         act_objid           = p_position
667         act_begda           = p_datum
668         act_endda           = p_datum
669         act_wegid           = 'ZAZ15'
670         act_plvar           = '01'
671         act_tdepth          = 2          " no recursion!

```

```

672     authority_check = ' '
673 TABLES
674     result_tab      = lt_acting_pos
675 EXCEPTIONS
676     no_plvar_found  = 01
677     no_entry_found  = 02.
678
679 IF sy-subrc NE 0. "Z15 not found
680 * Check for Z08 relationship (this is to person directly)
681 CALL FUNCTION 'RH_STRUC_GET'
682     EXPORTING
683         act_otype      = 'S'
684         act_objid      = p_position
685         act_begda      = p_datum
686         act_endda      = p_datum
687         act_wegid      = 'ZAZ08'
688         act_plvar      = '01'
689         act_tdepth     = 2           " no recursion!
690         authority_check = ' '
691 TABLES
692     result_tab      = lt_acting_per
693 EXCEPTIONS
694     no_plvar_found  = 01
695     no_entry_found  = 02.
696
697 * RT17935 - For Z08 persons, remove self reporting
698 IF sy-subrc EQ 0.
699     LOOP AT lt_acting_per
700         WHERE otype = 'P'.
701
702         IF gv_explain EQ 'X'.
703             PERFORM details_line TABLES gt_details_tab USING:
704                 p_position 'Is Acted By (Z08)'
705                 'P' lt_acting_per-objid.
706         ENDIF.
707
708 * Check that acting person is not the employee self
709 IF lt_acting_per-objid EQ p_pernr.
710     IF gv_explain EQ 'X'.
711         PERFORM details_line TABLES gt_details_tab USING:
712             space 'Ignored - self' space space.
713     ENDIF.
714     CONTINUE.
715 ENDIF.
716
717 * Get position of acting person
718 PERFORM get_position_of_person USING lt_acting_per-objid
719                                     p_datum
720                                     CHANGING lv_position.
721
722 IF gv_explain EQ 'X'.
723     PERFORM details_line TABLES gt_details_tab USING:
724         space 'Actor position' 'S' lv_position.
725 ENDIF.
726
727 IF lv_position IS NOT INITIAL.
728     p_position = lv_position.
729     p_acting = 'X'.
730 ENDIF.
731
732 ENDLOOP.
733 ENDIF.

```

```

733
734 * RT17935 - Else check that acting positions not exceptions
735 ELSE. "Z15 relationships found
736
737 LOOP AT lt_acting_pos
738 WHERE otype = 'S'.
739 IF gv_explain = 'X'.
740 PERFORM details_line TABLES gt_details_tab USING:
741 p_position 'Is Acted By (Z15)' 'S' lt_acting_pos-objid.
742 ENDIF.
743
744 * Check that acting position is not an exception position
745 IF lt_acting_pos-objid IN rng_ignpos.
746 IF gv_explain = 'X'.
747 PERFORM details_line TABLES gt_details_tab USING:
748 space 'Exception position!' 'S' lt_acting_pos-objid.
749 ENDIF.
750 CONTINUE.
751 ENDIF.
752
753 p_position = lt_acting_pos-objid.
754 p_acting = 'X'.
755 EXIT.
756 ENDLOOP.
757
758 ENDIF.
759
760 LOOP AT lt_acting_pos WHERE otype = 'S'.
761 IF lt_acting_pos-objid NOT IN rng_ignpos.
762 p_position = lt_acting_pos-objid.
763 EXIT.
764 ELSE.
765 IF gv_explain = 'X'.
766 PERFORM details_line TABLES gt_details_tab USING:
767 space 'Exception position!' 'S' lt_acting_pos-objid.
768 ENDIF.
769 ENDIF.
770 ENDLOOP.
771
772 ENDFORM. " CHECK_POSITION_ACTING
773 *&-----*
774 *& Form CONFIRM_POSITION
775 *&-----*
776 FORM confirm_position TABLES actor_tab TYPE tswhactor
777 USING p_pernr
778 p_position
779 p_datum.
780 REFRESH actor_tab.
781
782 * Ignore exception positions
783 IF p_position IN rng_ignpos.
784 gv_wfignore = 'X'.
785 IF gv_explain = 'X'.
786 PERFORM details_line TABLES gt_details_tab USING:
787 space 'Exception position!' 'S' p_position.
788 ENDIF.
789 RETURN.
790 ENDIF.
791
792 * Get holder of position
793 PERFORM get_holder TABLES actor_tab

```

```

794             USING p_position
795                 p_datum
796                 p_pernr.
797
798 LOOP AT actor_tab WHERE otype = 'P'.
799     IF gv_explain = 'X'.
800         PERFORM details_line TABLES gt_details_tab USING:
801             p_position 'Has holder' 'P' actor_tab-objid.
802     ENDIF.
803
804 * Check if actor employee is self
805     IF actor_tab-objid = p_pernr.
806         IF gv_explain = 'X'.
807             PERFORM details_line TABLES gt_details_tab USING:
808                 space 'Ignore - self' space space.
809         ENDIF.
810         DELETE actor_tab.
811     ENDIF.
812 ENDLOOP.
813
814 * If no holder found
815     IF sy-subrc NE 0.
816         IF gv_explain = 'X'.
817             PERFORM details_line TABLES gt_details_tab USING:
818                 p_position 'Ignore - no holder' space space.
819         ENDIF.
820     ENDIF.
821
822 ENDFORM.                                " CONFIRM_POSITION
823 *&-----*
824 *&      Form RECURSIVE_CALL
825 *&-----*
826 FORM recursive_call TABLES actor_tab
827     USING p_position
828         p_datum
829         p_maxlev
830     CHANGING p_acting
831             p_mgrinfo
832             p_callno.
833
834 DATA: act_cont TYPE swcont OCCURS 0 WITH HEADER LINE.
835
836 CLEAR: act_cont.
837 REFRESH: act_cont.
838
839 act_cont-element      = 'ORG OBJECT'.
840 act_cont-elemlength  = '025'.
841 act_cont-type         = 'C'.
842
843 CONCATENATE 'S' p_position INTO act_cont-value SEPARATED BY space.
844 APPEND act_cont.
845
846 CALL FUNCTION 'ZSWX_GET_DIRECT_MANAGER'
847     EXPORTING
848         ignore_acting      = gv_ignact
849         effective_date     = p_datum
850         explain            = gv_explain
851         max_levels         = p_maxlev
852     IMPORTING
853         acting             = p_acting
854     TABLES

```



```

855      actor_tab              = actor_tab
856      ac_container          = act_cont
857      CHANGING
858      manager_info           = p_mgrinfo
859      call_number             = p_callno
860      details_tab            = gt_details_tab
861      EXCEPTIONS
862      nobody_found            = 1
863      workflow_position_exception = 2
864      no_employee_position    = 3
865      no_orgunit_found        = 4
866      no_chiefpos_found       = 5
867      circular_reference      = 6
868      max_recursive           = 7
869      OTHERS                  = 8.
870
871      CASE sy-subrc.
872      WHEN 1.
873          RAISE nobody_found.
874      WHEN 2.
875          RAISE workflow_position_exeption.
876      WHEN 3.
877          RAISE no_employee_position.
878      WHEN 4.
879          RAISE no_orgunit_found.
880      WHEN 5.
881          RAISE no_chiefpos_found.
882      WHEN 6.
883          RAISE circular_reference.
884      WHEN 7.
885          RAISE max_recursive.
886      ENDCASE.
887
888
889      ENDFORM.                                " RECURSIVE_CALL
890      *&-----*
891      *&      Form RECURSIVE_CALL_ORG
892      *&-----*
893      FORM recursive_call_org
894          TABLES      actor_tab
895          USING         p_orgid
896                     p_datum
897                     p_maxlev
898          CHANGING     p_acting
899                     p_mgrinfo
900                     p_callno.
901
902      DATA: act_cont TYPE swcont OCCURS 0 WITH HEADER LINE.
903
904      CLEAR: act_cont.
905      REFRESH: act_cont.
906
907      act_cont-element      = 'ORG OBJECT'.
908      act_cont-elemlength = '025'.
909      act_cont-type         = 'C'.
910
911      CONCATENATE 'O' p_orgid INTO act_cont-value SEPARATED BY space.
912      APPEND act_cont.
913
914      CALL FUNCTION 'ZSWX_GET_DIRECT_MANAGER'
915          EXPORTING

```

```

916      ignore_acting      = gv_ignact
917      effective_date      = p_datum
918      explain             = gv_explain
919      max_levels          = p_maxlev
920      IMPORTING
921      acting              = p_acting
922      TABLES
923      actor_tab           = actor_tab
924      ac_container        = act_cont
925      CHANGING
926      manager_info        = p_mgrinfo
927      call_number         = p_callno
928      details_tab         = gt_details_tab
929      EXCEPTIONS
930      nobody_found        = 1
931      workflow_position_exception = 2
932      no_employee_position = 3
933      no_orgunit_found    = 4
934      no_chiefpos_found   = 5
935      circular_reference   = 6
936      max_recursive       = 7
937      OTHERS              = 8.

```

```

938
939 CASE sy-subrc.

```

```

940   WHEN 1.
941     RAISE nobody_found.
942   WHEN 2.
943     RAISE workflow_position_exeption.
944   WHEN 3.
945     RAISE no_employee_position.
946   WHEN 4.
947     RAISE no_orgunit_found.
948   WHEN 5.
949     RAISE no_chiefpos_found.
950   WHEN 6.
951     RAISE circular_reference.
952   WHEN 7.
953     RAISE max_recursive.

```

```

954   ENDCASE.

```

```

955
956
957 ENDFORM.

```

```

" RECURSIVE_CALL_ORG

```

```

958 *&-----*

```

```

959 *&      Form  GET_POSITION_OF_PERSON

```

```

960 *&-----*

```

```

961 FORM get_position_of_person USING      p_pernr
962                                     p_datum
963                                     CHANGING p_position.

```

```

964
965 DATA BEGIN OF lt_postab OCCURS 0.
966 INCLUDE STRUCTURE swhactor.
967 DATA END OF lt_postab.

```

```

968
969 CLEAR p_position.

```

```

970
971 CLEAR lt_postab.
972 REFRESH lt_postab.

```

```

973
974 CALL FUNCTION 'RH_STRUC_GET'
975   EXPORTING
976     act_otype      = 'P'

```

```

977      act_objid      = p_pernr
978      act_begda      = p_datum
979      act_endda      = p_datum
980      act_wegid      = 'B008'
981      act_plvar      = '01'
982      act_tdepth     = 2          " no recursion!
983      authority_check = ' '
984  TABLES
985      result_tab      = lt_postab
986  EXCEPTIONS
987      no_plvar_found  = 01
988      no_entry_found  = 02.
989
990  LOOP AT lt_postab WHERE otype = 'S'.
991      p_position = lt_postab-objid.
992      EXIT.
993  ENDLOOP.
994
995  ENDFORM.                                " GET_POSITION_OF_PERSON
996  *&-----*
997  *&      Form  GET_CHIEF_HIERARCHY
998  *&-----*
999  FORM get_chief_hierarchy TABLES p_actor_tab
1000                                USING p_position
1001                                p_datum
1002                                p_depth.
1003
1004  DATA: p_orgunit_tab TYPE swfactor OCCURS 10 WITH HEADER LINE.
1005
1006
1007  CLEAR p_orgunit_tab.
1008  REFRESH p_orgunit_tab.
1009  eval_path = 'S-O-O'.                    " position->org-unit->org-unit
1010  eval_depth = p_depth. "0. CP 16.03.2019
1011  *   Get all orgs upwards in the hierarchy
1012  CALL FUNCTION 'RH_STRUC_GET'
1013      EXPORTING
1014          act_otype      = 'S'
1015          act_objid      = p_position
1016          act_begda      = p_datum
1017          act_endda      = p_datum
1018          act_wegid      = eval_path
1019          act_plvar      = '01'
1020          act_tdepth     = eval_depth " recursion check!
1021          authority_check = ' '
1022  TABLES
1023      result_tab      = p_orgunit_tab
1024  EXCEPTIONS
1025      no_plvar_found  = 01
1026      no_entry_found  = 02.
1027
1028  IF sy-subrc NE 0.
1029      MESSAGE e010 WITH 'Cannot find orgunits above position'
1030                      p_position space space
1031                      RAISING no_orgunit_found.
1032      RETURN.
1033  ENDIF.
1034
1035  IF gv_explain EQ 'X'.
1036      LOOP AT p_orgunit_tab.
1037          IF sy-tabix = 1.

```

```

1038         PERFORM details_line TABLES gt_details_tab USING:
1039             p_position 'Belongs to Org' 'O' p_orgunit_tab-objid.
1040     ELSE.
1041         PERFORM details_line TABLES gt_details_tab USING:
1042             space 'Belongs to Org' 'O' p_orgunit_tab-objid.
1043     ENDIF.
1044 ENDLOOP.
1045 ENDIF.
1046
1047     PERFORM get_chief_positions TABLES p_orgunit_tab
1048                                     p_actor_tab
1049                                     USING p_datum.
1050
1051
1052
1053
1054 ENDFORM.                                " GET_CHIEF_HIERARCHY
1055 *&-----*
1056 *&      Form  DETAILS_LINE
1057 *&-----*
1058 FORM details_line TABLES details_tab
1059                     USING p_key
1060                         p_item
1061                         p_otype
1062                         p_value.
1063
1064 DATA: lv_details TYPE text80.
1065 DATA: lv_desc(40).
1066
1067 lv_details(8) = p_key.
1068 lv_details+10(30) = p_item.
1069 lv_details+40(8) = p_value.
1070 CLEAR lv_desc.
1071 CASE p_otype.
1072     WHEN 'S'.
1073         PERFORM get_object_name USING 'S' p_value CHANGING lv_desc.
1074     WHEN 'O'.
1075         PERFORM get_object_name USING 'O' p_value CHANGING lv_desc.
1076     WHEN 'P'.
1077         PERFORM get_employee_name USING p_value CHANGING lv_desc.
1078 ENDCASE.
1079 lv_details+50 = lv_desc.
1080 APPEND lv_details TO details_tab.
1081
1082 ENDFORM.                                " DETAILS_LINE
1083 *&-----*
1084 *&      Form  GET_OBJECT_NAME
1085 *&-----*
1086 FORM get_object_name USING p_otype
1087                         p_objid
1088                         CHANGING p_stext.
1089
1090 SELECT SINGLE stext INTO p_stext
1091 FROM hrp1000
1092 WHERE plvar = '01'
1093     AND otype = p_otype
1094     AND objid = p_objid
1095     AND istat = '1'
1096     AND begda LE gv_effdate
1097     AND endda GE gv_effdate.
1098

```

```

1099 ENDFORM.                                " GET_OBJECT_NAME
1100 *&-----*
1101 *&      Form GET_EMPLOYEE_NAME
1102 *&-----*
1103 FORM get_employee_name USING p_pernr
1104      CHANGING p_ename.
1105
1106 SELECT SINGLE ename INTO p_ename
1107 FROM pa0001
1108 WHERE pernr = p_pernr
1109      AND sprps = space
1110      AND begda LE gv_effdate
1111      AND endda GE gv_effdate.
1112
1113 ENDFORM.                                " GET_EMPLOYEE_NAME
1114 *&-----*
1115 *&      Form GET_CONTRACTOR_SUPERVISOR
1116 *&-----*
1117 FORM get_contractor_supervisor USING p_pernr
1118      p_datum
1119      CHANGING p_position.
1120
1121 DATA: lv_stext LIKE hrp1000-stext.
1122
1123 DATA: actor_tab TYPE tswhactor WITH HEADER LINE.
1124
1125 CLEAR p_position.
1126
1127 SELECT zzconpos
1128 INTO p_position
1129 FROM pa0016
1130 WHERE pernr = p_pernr
1131      AND sprps EQ space
1132      AND begda LE p_datum
1133      AND endda GE p_datum
1134      AND zzconpos NE '00000000'
1135 ORDER BY aedtm.
1136 ENDSELECT.
1137 IF sy-subrc EQ 0.
1138 * Check position exists
1139 SELECT SINGLE stext INTO lv_stext
1140 FROM hrp1000
1141 WHERE plvar = '01'
1142      AND otype = 'S'
1143      AND objid = p_position
1144      AND istat = '1'
1145      AND begda LE p_datum
1146      AND endda GE p_datum.
1147 IF sy-subrc NE 0.
1148 CLEAR p_position.
1149 RETURN.
1150 ENDIF.
1151 ELSE. "No IT0016 record
1152 CLEAR p_position.
1153 RETURN.
1154 ENDIF.
1155
1156 * Check for active holder
1157 * Get holder of position
1158 REFRESH actor_tab.
1159 PERFORM get_holder TABLES actor_tab

```

```
1160             USING p_position
1161                 p_datum
1162                 p_pernr.
1163
1164 LOOP AT actor_tab WHERE otype = 'P'.
1165     IF gv_explain = 'X'.
1166         PERFORM details_line TABLES gt_details_tab USING:
1167             p_position 'Has holder' 'P' actor_tab-objid.
1168     ENDIF.
1169
1170 * Check if actor employee is self
1171     IF actor_tab-objid = p_pernr.
1172         IF gv_explain = 'X'.
1173             PERFORM details_line TABLES gt_details_tab USING:
1174                 space 'Ignore - self' space space.
1175         ENDIF.
1176         DELETE actor_tab.
1177     ELSE.
1178         RETURN.
1179     ENDIF.
1180
1181 ENDLOOP.
1182
1183 * If no holder found
1184 IF sy-subrc NE 0.
1185     IF gv_explain = 'X'.
1186         PERFORM details_line TABLES gt_details_tab USING:
1187             p_position 'Ignore - no holder' space space.
1188         CLEAR p_position.
1189     ENDIF.
1190 ENDIF.
1191
1192 ENDFORM.
```

Subroutines:		Line	Page
CHECK_ACTING_MANAGER.....	LZSWXAF02	606	52
CHECK_ACTING_POSITION.....	LZSWXAF02	394	49
	LZSWXAF02	570	52
CHECK_POSITION_ACTING.....	LZSWXAF02	650	53
CONFIRM_POSITION.....	LZSWXAF02	776	55
DETAILS_LINE.....	LZSWXAF02	377	49
	LZSWXAF02	703	54
	LZSWXAF02	711	54
	LZSWXAF02	722	54
	LZSWXAF02	740	55
	LZSWXAF02	747	55
	LZSWXAF02	766	55
	LZSWXAF02	786	55
	LZSWXAF02	800	56
	LZSWXAF02	807	56
	LZSWXAF02	817	56
	LZSWXAF02	1038	60
	LZSWXAF02	1041	60
	LZSWXAF02	1058	60
	LZSWXAF02	1166	62
	LZSWXAF02	1173	62
	LZSWXAF02	1186	62
GET_CHIEF_HIERARCHY.....	LZSWXAF02	999	59
GET_CHIEF_POSITIONS.....	LZSWXAF02	346	48
	LZSWXAF02	1047	60
GET_CONTRACTOR_SUPERVISOR.....	LZSWXAF02	1117	61
GET_EE_DATA.....	LZSWXAF02	12	43
GET_EMPLOYEE_NAME.....	LZSWXAF02	1077	60
	LZSWXAF02	1103	61
GET HOLDER.....	LZSWXAF02	260	47
	LZSWXAF02	482	50
	LZSWXAF02	595	52
	LZSWXAF02	793	55
	LZSWXAF02	1159	61
GET_OBJECT_NAME.....	LZSWXAF02	1073	60
	LZSWXAF02	1075	60
	LZSWXAF02	1086	60
GET_ORG_UNITS.....	LZSWXAF02	311	48
GET_POSITION_OF_PERSON.....	LZSWXAF02	718	54
	LZSWXAF02	961	58
GET_REPORTS_TO_POSITION.....	LZSWXAF02	197	46
	LZSWXAF02	547	51
GET_STAT2.....	LZSWXAF02	293	47
	LZSWXAF02	635	53
GET_VALIDATE_REPORTS_TO.....	LZSWXAF02	537	51
RECURSIVE_CALL.....	LZSWXAF02	826	56
RECURSIVE_CALL_ORG.....	LZSWXAF02	893	57

Messages:		Line	Page
WZ010.....	LZSWXAF02	124	45
	LZSWXAF02	132	45
	LZSWXAF02	167	45
	LZSWXAF02	1029	59

External calls

Function modules:

		Line	Page
RH_STRUC_GET.....	LZSWXAF02	146	45
	LZSWXAF02	204	46
	LZSWXAF02	271	47
	LZSWXAF02	320	48
	LZSWXAF02	359	48
	LZSWXAF02	420	49
	LZSWXAF02	439	50
	LZSWXAF02	663	53
	LZSWXAF02	681	54
	LZSWXAF02	974	58
	LZSWXAF02	1012	59
	LZSWXAF02		
ZSWX_GET_DIRECT_MANAGER.....	LZSWXAF02	846	56
	LZSWXAF02	914	57
Z_GET_COMPANY_CODE_GROUPING...	LZSWXAF02	52	43

DATABASE OPERATIONS

SELECT:		Line	Page
HRP1000.....	LZSWXAF02	1091	60
	LZSWXAF02	1140	61
HRP1001.....	LZSWXAF02	86	44
	LZSWXAF02	110	44
PA0000.....	LZSWXAF02	640	53
PA0001.....	LZSWXAF02	180	45
	LZSWXAF02	1107	61
PA0016.....	LZSWXAF02	1129	61
PA0105.....	LZSWXAF02	42	43

INTERNAL TABLE OPERATIONS

READ TABLE:	Line	Page
LT_REPORTS_TO_TAB..... LZSWXAF02	244	46

DELETE:	Line	Page
ACTING_TAB..... LZSWXAF02	469	50
ACTOR_TAB..... LZSWXAF02	810	56
..... LZSWXAF02	1176	62
ADD_ACTOR_TAB..... LZSWXAF02	459	50
LT_HOLDER_TAB..... LZSWXAF02	290	47

```

1  *-----*
2  ***INCLUDE LZSWXAF03 .
3  *-----*
4  *&-----*
5  *&      Form  FILTER_BY_COMPANY_GROUPING
6  *&-----*
7  *      text
8  *-----*
9  *      —>P_ACT_TAB  text
10 *-----*
11 FORM filter_by_company_grouping  TABLES  p_act_tab STRUCTURE zppa013.
12
13
14 DATA: lv_index LIKE sy-tabix,
15        ls_zppa000 LIKE zppa000.
16
17 LOOP AT p_act_tab.
18     lv_index = sy-tabix.
19
20 * Get email address
21     SELECT SINGLE usrid_long
22           FROM pa0105
23           INTO p_act_tab-email
24           WHERE pernr EQ p_act_tab-pernr
25           AND   subty EQ '0006'
26           AND   begda LE sy-datum
27           AND   endda GE sy-datum.
28
29 * Get org data
30     SELECT SINGLE * FROM pa0001
31           WHERE pernr EQ p_act_tab-pernr
32           AND   begda LE sy-datum
33           AND   endda GE sy-datum.
34     CALL FUNCTION 'Z_GET_COMPANY_CODE_GROUPING'
35       EXPORTING
36         pernr           = pa0001-pernr
37         datum           = sy-datum
38         werks           = pa0001-werks
39       IMPORTING
40         oszppa000       = ls_zppa000
41       EXCEPTIONS
42         no_company_code_group_found = 1
43         OTHERS              = 2.
44
45     IF ls_zppa000-ccgrp NE gv_users_ccgrp.
46         DELETE p_act_tab INDEX lv_index.
47         CONTINUE.
48     ENDIF.
49
50 ENDLOOP.
51
52 ENDFORM.              " FILTER_BY_COMPANY_GROUPING
53 *&-----*
54 *&      Form  GET_USERS_CCGRP
55 *&-----*
56 *      text
57 *-----*
58 *      <—P_GV_CCGRP  text
59 *-----*
60 FORM get_users_ccgrp  CHANGING p_ccgrp.
61

```

```

62  DATA: ls_zppa000 LIKE zppa000,
63          lv_pernr LIKE pa0105-pernr.
64
65  CLEAR p_ccgrp.
66
67  SELECT SINGLE pernr FROM pa0105
68                      INTO lv_pernr
69                      WHERE subty = '0001'
70                      AND   begda LE sy-datum
71                      AND   endda GE sy-datum
72                      AND   usrid = sy-uname.
73
74  CALL FUNCTION 'Z_GET_COMPANY_CODE_GROUPING'
75      EXPORTING
76          pernr           = lv_pernr
77          datum           = sy-datum
78      IMPORTING
79          oszppa000       = ls_zppa000
80      EXCEPTIONS
81          no_company_code_group_found = 1
82          OTHERS           = 2.
83
84  p_ccgrp = ls_zppa000-ccgrp.
85
86  ENDFORM.                                " GET_USERS_CCGRP
87  *&-----*
88  *&      Form  CHECK_CHIEF_POSITIONS
89  *&-----*
90  *      text
91  *-----*
92  *      -->P_CHIEF_POSITION_TAB  text
93  *      -->P_ACTOR_TAB  text
94  *-----*
95  FORM check_chief_positions
96      TABLES  p_chief_position_tab STRUCTURE swfactor
97              p_actor_tab STRUCTURE swfactor
98      USING    p_acting_vac_flag
99              p_pernr
100             p_datum
101  CHANGING    p_acting
102             p_reports_to_position.
103
104  DATA BEGIN OF lt_orgunit_tab OCCURS 0.
105      INCLUDE STRUCTURE swfactor.
106  DATA END OF lt_orgunit_tab.
107
108  DATA BEGIN OF lt_chief_position_tab OCCURS 0.
109      INCLUDE STRUCTURE swfactor.
110  DATA END OF lt_chief_position_tab.
111
112  LOOP AT p_chief_position_tab.
113
114      IF ( p_acting_vac_flag = '1' OR
115          p_acting_vac_flag = '2' )
116          and gv_ignact is initial.
117
118  *BEGIN HCC 03.04.2014
119  *see if the position have a inform to relation.
120      IF NOT p_reports_to_position IS INITIAL.
121          DATA: acting_tab TYPE TABLE OF swfactor.
122          CALL FUNCTION 'RH_STRUC_GET'

```



```

183             p_datum
184             p_pernr.
185 *   LOOP AT p_actor_tab WHERE objid = p_pernr.
186 *   DELETE p_actor_tab INDEX sy-tabix.
187 *   ENDLOOP.
188
189 * Check acting
190   PERFORM check_acting_manager TABLES p_actor_tab
191   USING p_pernr
192   p_datum
193   CHANGING p_acting.
194
195 *BEGIN HCC 03.04.2014
196 * Remove the user themselves
197   DATA: lv_own_user.
198   CLEAR lv_own_user.
199   LOOP AT p_actor_tab WHERE objid = p_pernr.
200     lv_own_user = 'Y'.
201   ENDLOOP.
202   IF lv_own_user = 'Y'.
203     DELETE p_actor_tab WHERE objid = p_pernr.
204   ENDIF.
205
206 *END HCC 03.04.2014
207
208   DESCRIBE TABLE p_actor_tab LINES counter.
209   IF counter > 0.
210     EXIT.
211   ENDIF.
212
213 *-----*
214 * Only for acting vacant flag '2', check the "Reports To" position
215 * before checking the next chief position
216 *-----*
217   IF p_acting_vac_flag = '2'.
218 * Get and validate "Reports To" Position
219   PERFORM get_validate_reports_to TABLES p_actor_tab
220   USING p_chief_position_tab-objid
221   p_acting_vac_flag
222   p_pernr
223   p_datum
224   CHANGING p_reports_to_position
225   p_acting.
226
227   DESCRIBE TABLE p_actor_tab LINES counter.
228   IF counter > 0.
229 * Reports to position found and has a holder - this is the approver
230   EXIT.
231   ELSE.
232     IF NOT p_reports_to_position IS INITIAL.
233 * Empty reports to position found - go back the start of org branch
234 * with this position
235
236     PERFORM get_org_units TABLES lt_orgunit_tab
237     USING p_reports_to_position
238     p_datum.
239 *** 4. Find the manager of the org-units or a superior org-unit.
240     PERFORM get_chief_positions TABLES lt_orgunit_tab
241     lt_chief_position_tab
242     USING p_datum.
243

```

```
244 *      Check chief positions
245      PERFORM check_chief_positions TABLES lt_chief_position_tab
246      p_actor_tab
247      USING p_acting_vac_flag
248      p_pernr
249      p_datum
250      CHANGING p_acting
251      p_reports_to_position.
252      DESCRIBE TABLE p_actor_tab LINES counter.
253      IF counter > 0.
254 * Reports to position found and has a holder - this is the approver
255      EXIT.
256      ENDIF.
257
258      ENDIF.
259      ENDIF.
260      ENDIF.
261 *-----*
262
263      ENDLOOP.      " at chiefs tab
264
265      ENDFORM.      " CHECK_CHIEF_POSITIONS
```

Subroutines:		Line	Page
CHECK_ACTING_MANAGER.....	LZSWXAF03	190	70
CHECK_ACTING_POSITION.....	LZSWXAF03	168	69
CHECK_CHIEF_POSITIONS.....	LZSWXAF03	95	68
	LZSWXAF03	245	71
FILTER BY COMPANY GROUPING....	LZSWXAF03	11	67
GET_CHIEF_POSITIONS.....	LZSWXAF03	240	70
GET_HOLDER.....	LZSWXAF03	181	69
GET_ORG_UNITS.....	LZSWXAF03	236	70
GET_USERS_CCGRP.....	LZSWXAF03	60	67
GET_VALIDATE_REPORTS_TO.....	LZSWXAF03	219	70

External calls

Function modules:	Line	Page
HR_STRUC_GET..... LZSWXAF03	122	68
LZSWXAF03	148	69
Z_GET_COMPANY_CODE_GROUPING... LZSWXAF03	34	67
LZSWXAF03	74	68

DATABASE OPERATIONS

SELECT:		Line	Page
PA0001.....	LZSWXAF03	30	67
PA0105.....	LZSWXAF03	22	67
	LZSWXAF03	67	68

INTERNAL TABLE OPERATIONS

READ TABLE:	Line	Page
REQUESTER_TAB..... LZSWXAF03	163	69

DELETE:	Line	Page
P_ACTOR_TAB..... LZSWXAF03	203	70
P_ACT_TAB..... LZSWXAF03	46	67

```

1  *-----
2  ***INCLUDE LSWXAO01 .
3  *-----
4  *&-----*
5  *&      Module  STATUS_051  OUTPUT
6  *&-----*
7  *      text
8  *-----*
9  module status_051 output.
10     perform dynpro_051_init.
11 endmodule.          " STATUS_051  OUTPUT
12 *&-----*
13 *&      Module  SCREEN_011  OUTPUT
14 *&-----*
15 *      text
16 *-----*
17 module screen_011 output.
18     perform screen_011.
19 endmodule.          " SCREEN_011  OUTPUT
20 *&-----*
21 *&      Module  dynpro_010_init  OUTPUT
22 *&-----*
23 *      text
24 *-----*
25 module dynpro_010_init output.
26     perform dynpro_010_init.
27 endmodule.          " dynpro_010_init  OUTPUT
28 *&-----*
29 *&      Module  dynpro_050_init  OUTPUT
30 *&-----*
31 *      text
32 *-----*
33 module dynpro_050_init output.
34     perform dynpro_050_init.
35 endmodule.          " dynpro_050_init  OUTPUT
36 *&-----*
37 *&      Module  dynpro_011_init  OUTPUT
38 *&-----*
39 *      text
40 *-----*
41 module dynpro_011_init output.
42     perform dynpro_011_init.
43 endmodule.          " dynpro_011_init  OUTPUT

```



```
1  *-----
2  ***INCLUDE LSWXAI01 .
3  *-----
4  *&-----*
5  *&      Module  ABORT  INPUT
6  *&-----*
7  *      text
8  *-----*
9  module abort input.
10   perform abort.
11 endmodule.              " ABORT  INPUT
12 *&-----*
13 *&      Module  FORM_PROCESS  INPUT
14 *&-----*
15 *      text
16 *-----*
17 module form_process input.
18   perform form_process.
19 endmodule.              " FORM_PROCESS  INPUT
```

PAI modules:		Line	Page
ABORT.....	LZSWXAI01	9	78
FORM_PROCESS.....	LZSWXAI01	17	78

Subroutines:		Line	Page
ABORT.....	LZSWXAI01	10	78
FORM_PROCESS.....	LZSWXAI01	18	78